



What Will We be Doing?

- Creating a multiplayer game using Photon.
- Setting up a lobby to get players together before playing.
- Creating a player controller which can move, jump and steal the hat from other players.
- Using Unity's UI system to create the menu and in-game UI.

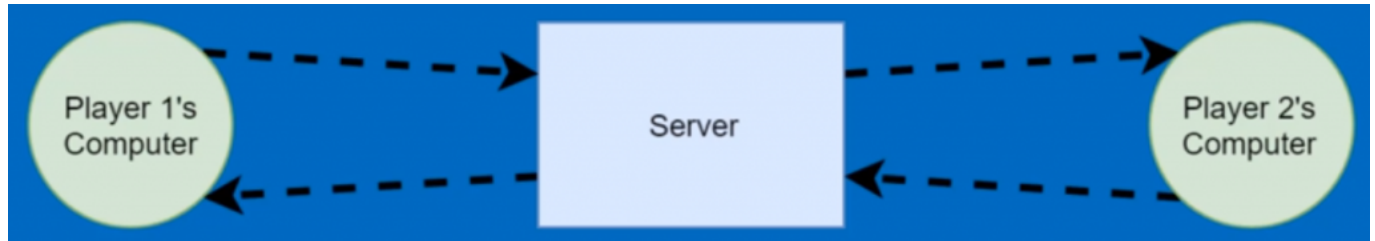


What's Photon?

Photon is a networking framework for Unity. It's very similar in structure to UNet (now deprecated).

How Does Multiplayer Work?

Here's how multiplayer in Photon and many other games works.



Players connect to a server and send / receive messages. These can be updates to other players positions, when a player shoots, when you get hit, etc. If there's things you want other players on the server to know that is not being done directly on their computer, it will be sent through the network.

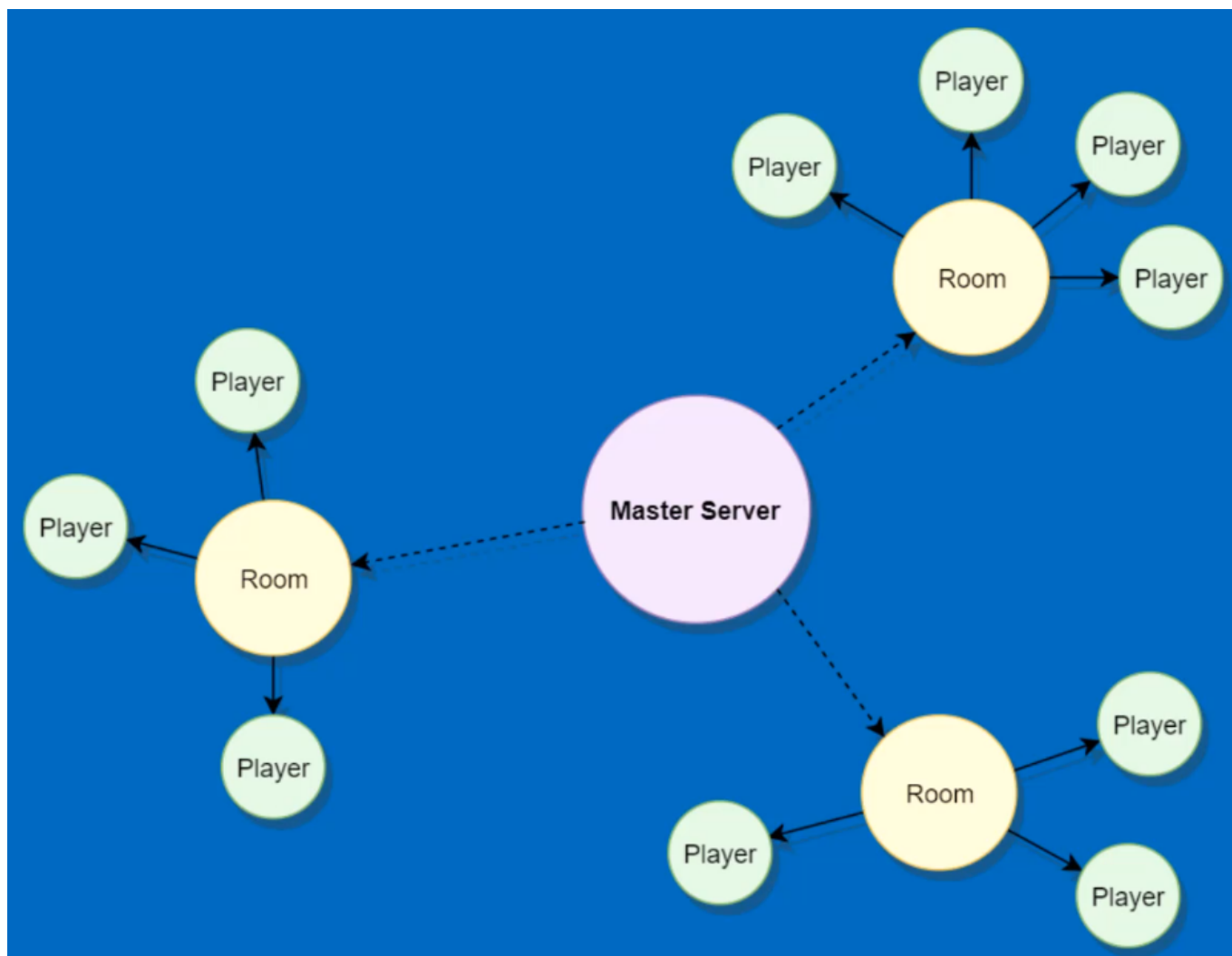
For example, if player 1 shoots their gun, that message will be sent to the server and then received by player 2 on their computer. They will then show player 1 shooting their gun and spawning a bullet.

By default in Photon, messages can be sent up to 20 times a second.

Master Server

A master server is a server on Photon for each game, or even unique versions of a game. This is where all the players and rooms for that game are located. Players can view all the existing rooms, join them or create a new one. You can only see rooms from the master server you're in. You cannot join a room of another game.

By default, Photon can allow up to 20 players at once connected to the server. This can be upgraded on their site. They also feature servers all around the world.

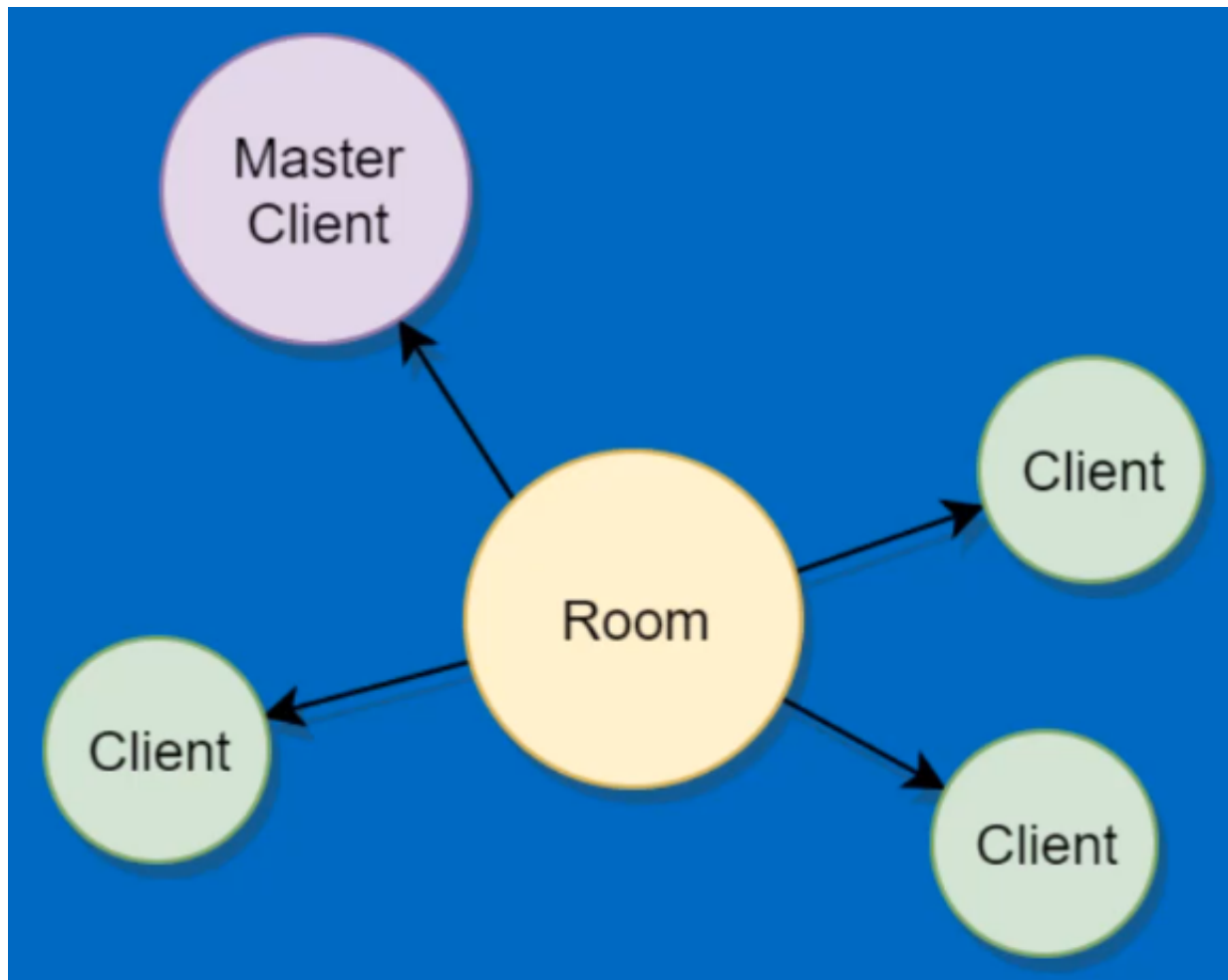


Rooms

Think of a room like a match or lobby. This is where a group of players can send messages to each other. Sync values, positions, rotations, animations, etc.

Players

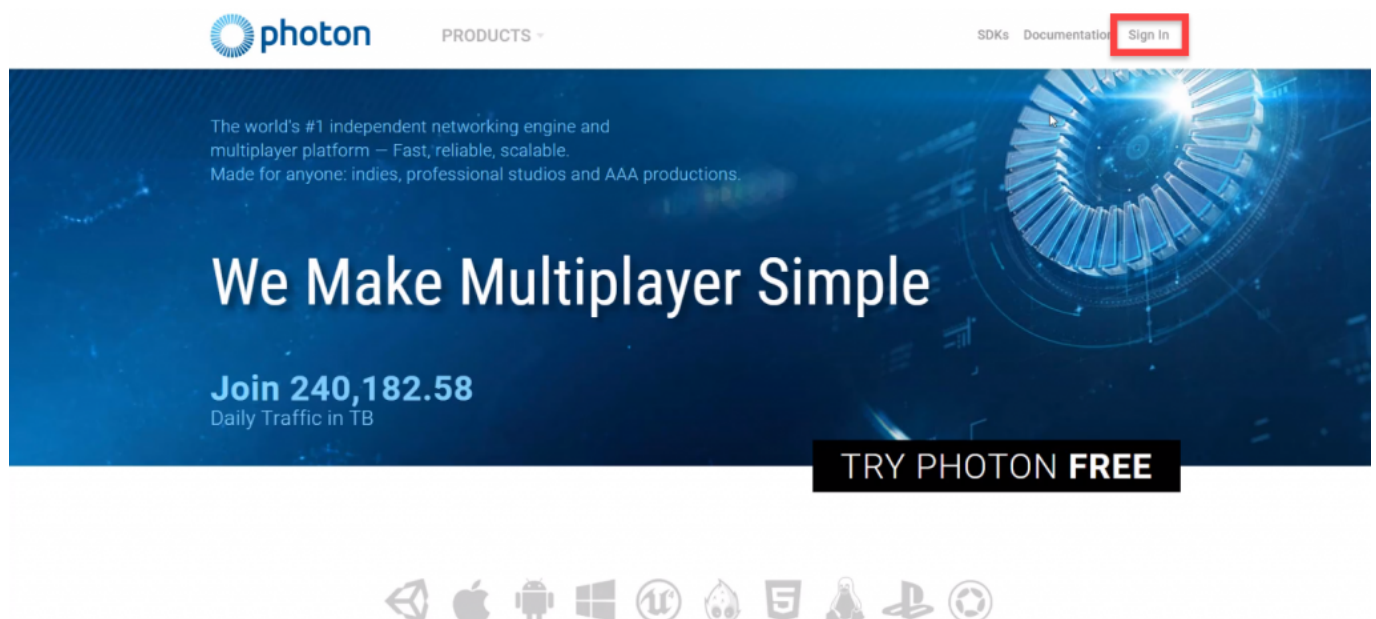
Players (aka clients) are the people that populate rooms. Each room has a master client (host). This is by default the player who creates the room and is useful for checking and running server side things.





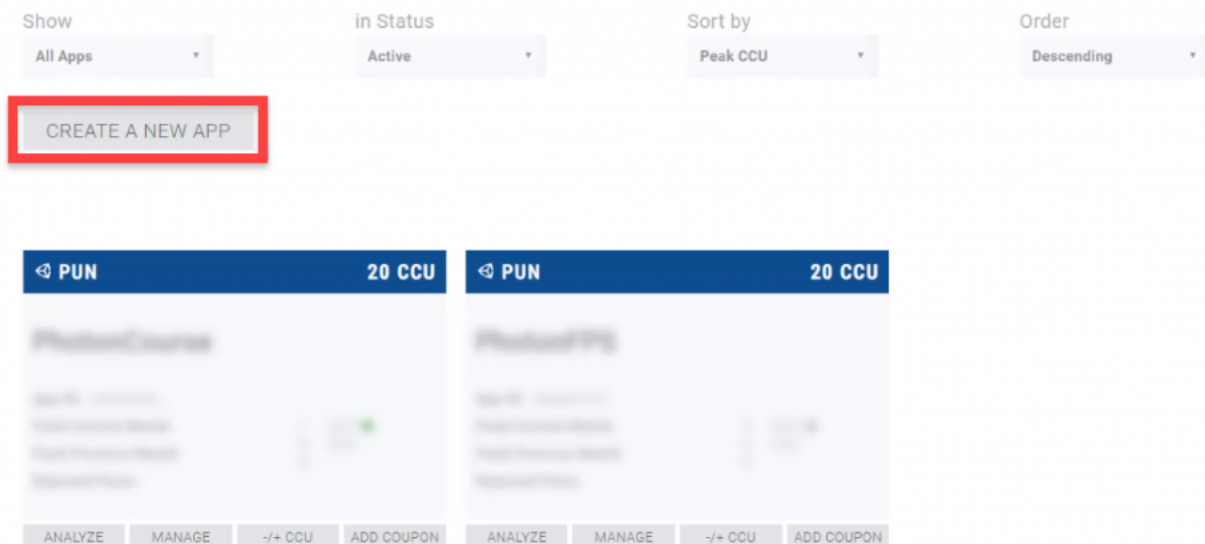
Setting up Photon

Before we can use Photon, we need to get an App Id. To do this, go to www.photonengine.com and click on the **Sign In** button. Either sign in or create a new account.



Once you sign in, you'll be taken to the applications page. Here, we want to click on the **CREATE A NEW APP** button.

Your Photon Cloud Applications



Here, we can fill out the info for our new application.

- Set **Photon Type** to *Photon PUN*
- Enter a name for your app

Then click on the **CREATE** button.



Create a New Application

The application defaults to the **Free Plan**.
You can change the plan at any time.

Photon Type *

Photon PUN

Name *

MyPhotonApp

Description

Short description, 1024 chars max.

Url

http://enter.your-url.here/

CREATE

or [go back to the application list.](#)

This will take you back to the applications page. Here, you want to find your app and copy the **App ID**. This is how Photon will know what your game/app is and to keep track of analytics.

 **PUN** 20 CCU

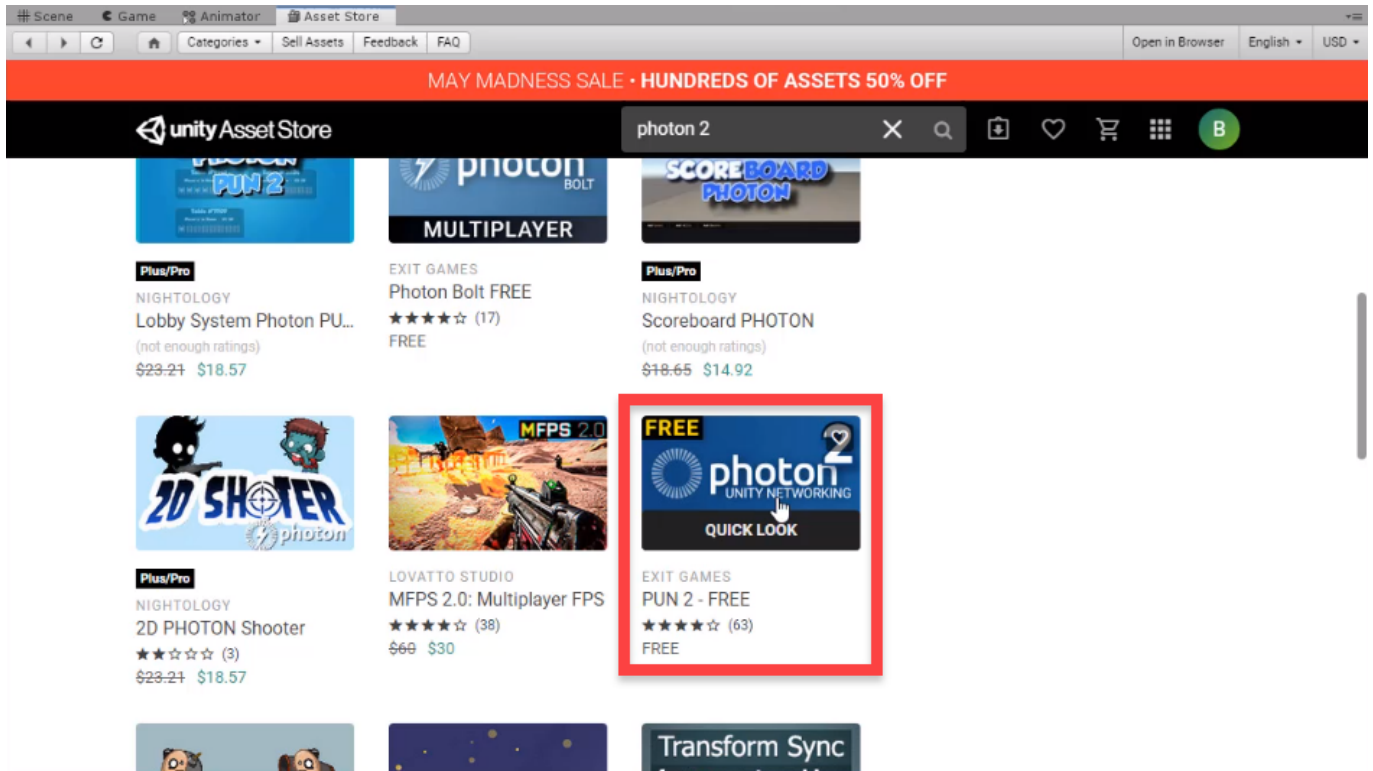
MyPhotonApp

App ID: 9ffa9af3-

Peak Current Month	0	CCU	→
Peak Previous Month	0	CCU	○
Rejected Peers	0		○

Importing Photon into Unity

In Unity, create a new project and open up the **Asset Store** window (*Window > Asset Store*). Search for “photon 2” and download the **PUN 2 - FREE** asset.



When the import package window pops up, we're going to disable some parts of the asset since we don't need them.

- Photon > PhotonChat
- Photon > PhotonRealtime > Demos
- Photon > PhotonUnityNetwork > Demos

Click **Import**, then once that's done the **PUN Wizard** window should open. Here, we just want to paste in our app id and click **Setup Project**.

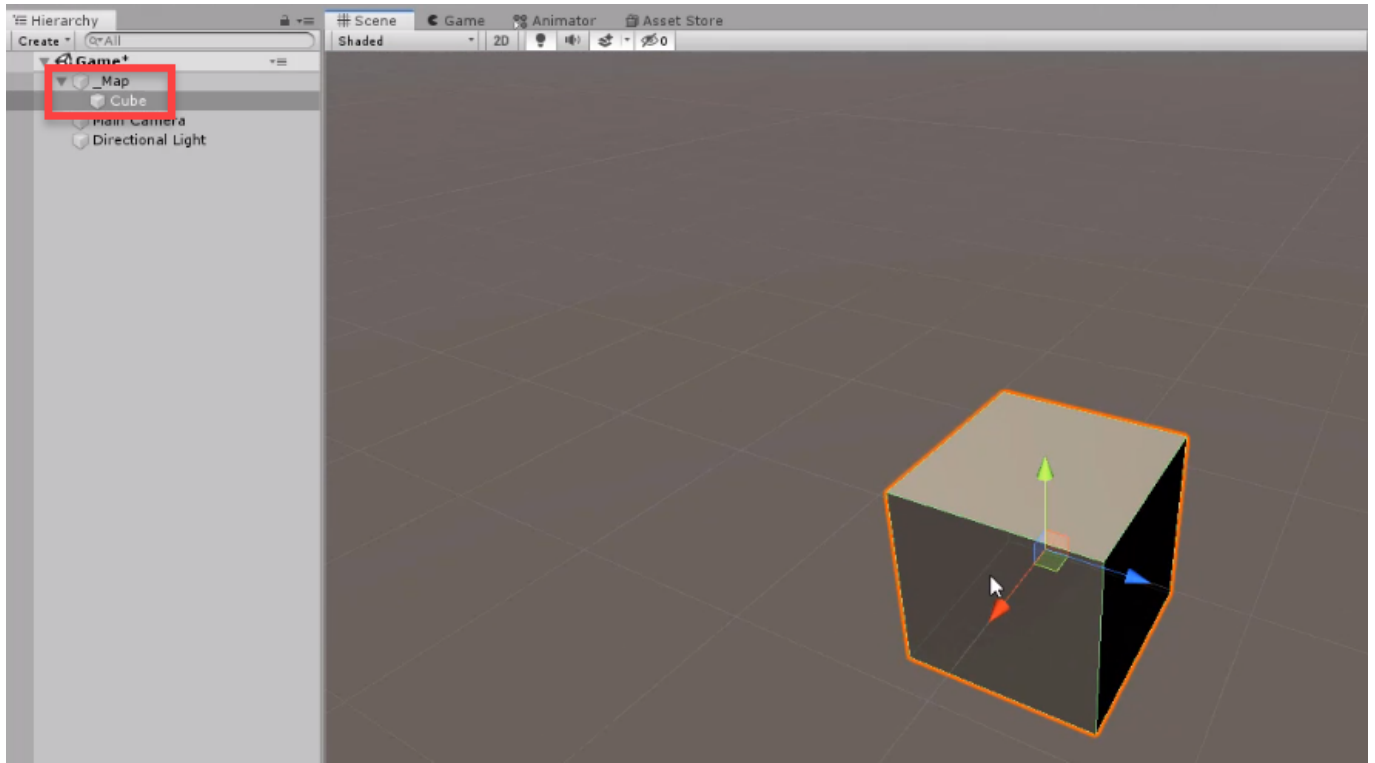


Now Photon's installed and ready to go!

Game Scene

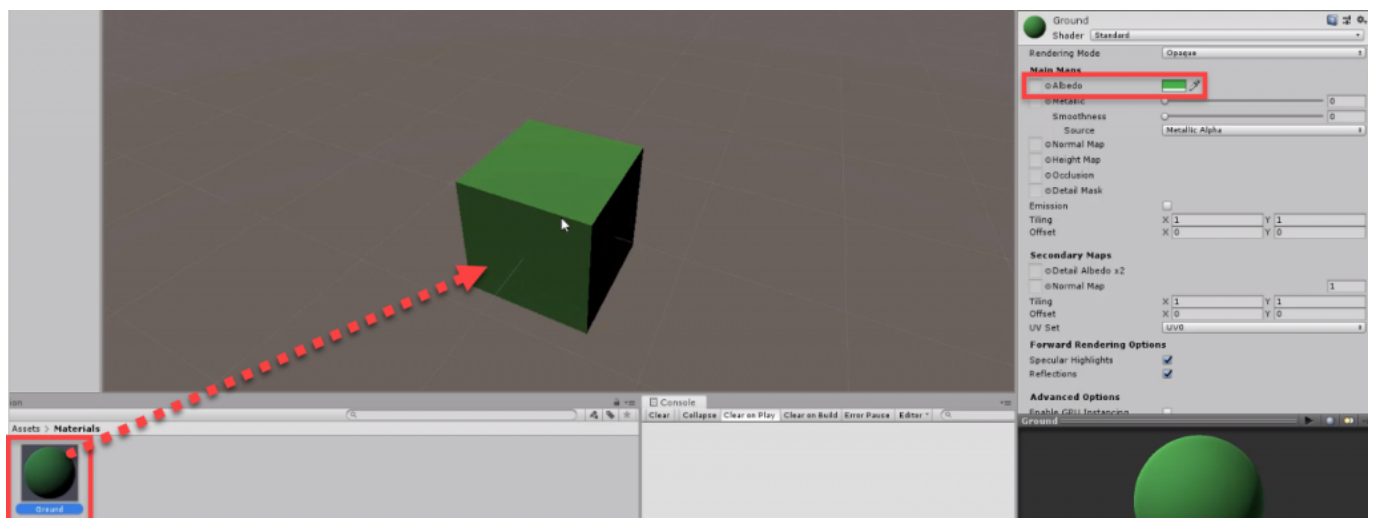
Create 2 new scenes (Game and Menu). Open up the **Game** scene, as we're going to be developing our level.

To begin, create a new empty GameObject (right click Hierarchy > *Create Empty*) and call it **_Map**. As a child of that, create a new cube object.



This is going to be our ground. Create a new Material called **Ground** and apply it to the cube.

- Set the **Albedo** color to green
- Set the **Smoothness** to 0

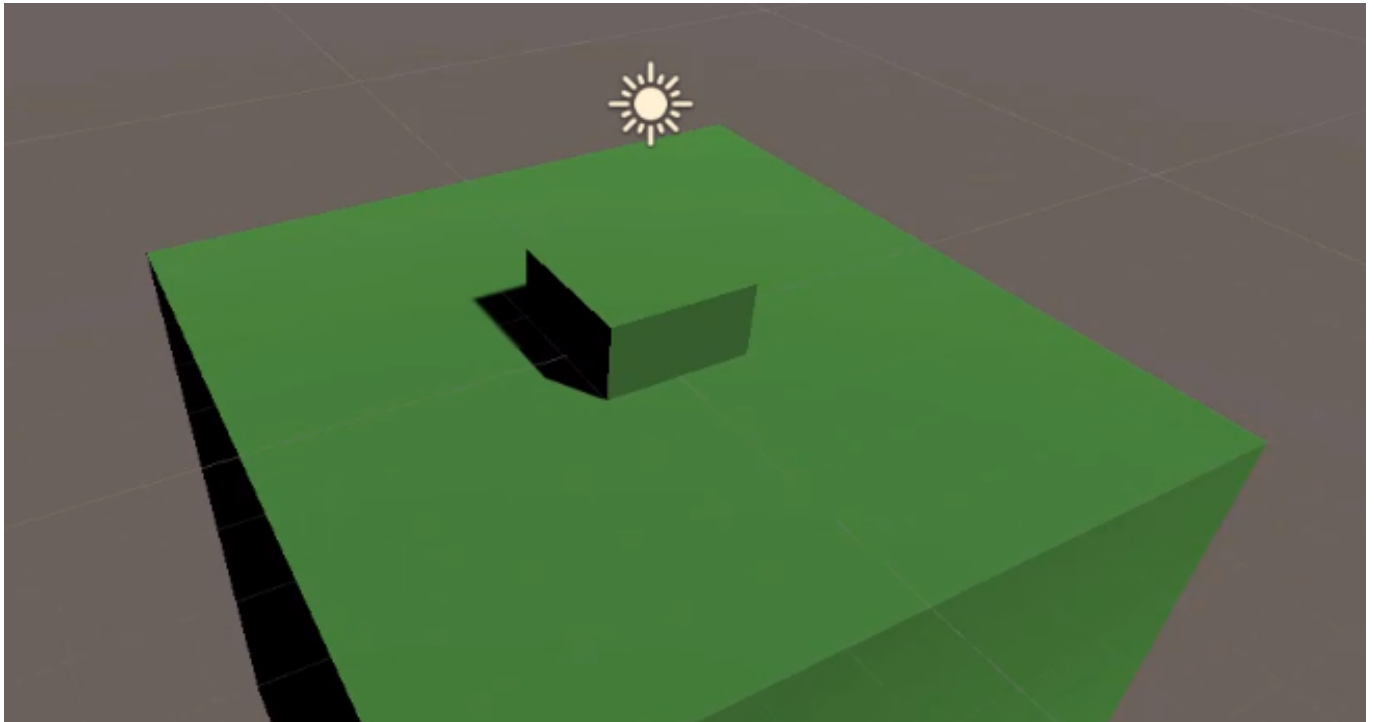


Resize the cube.



- Set the **Y Position** to -5
- Set the **Scale** to 15 (not seen in image)

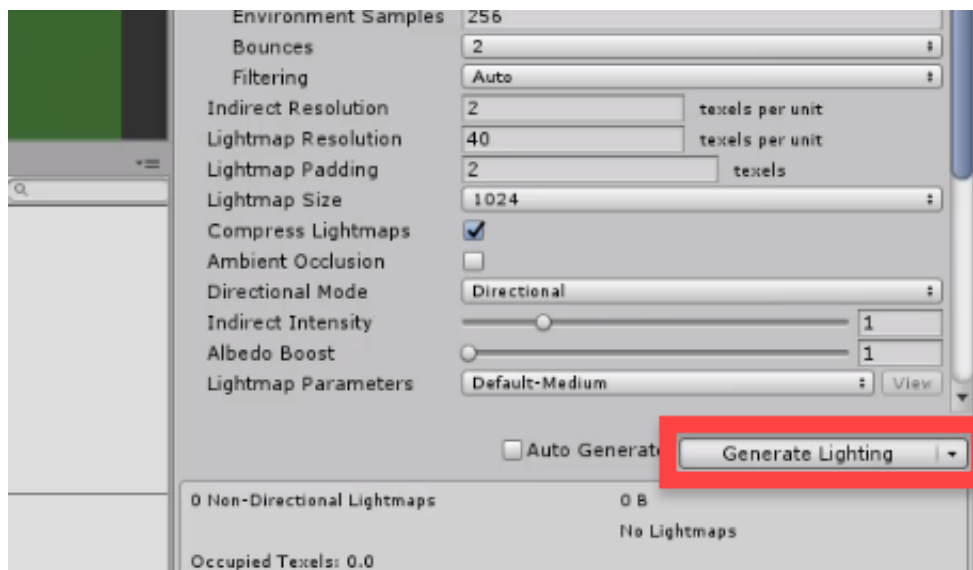
Now duplicate (Ctrl + D) the cube and resize/position it so it's a little platform in the middle like below.



For this game, we're going to have an isometric camera, so let's change that now.

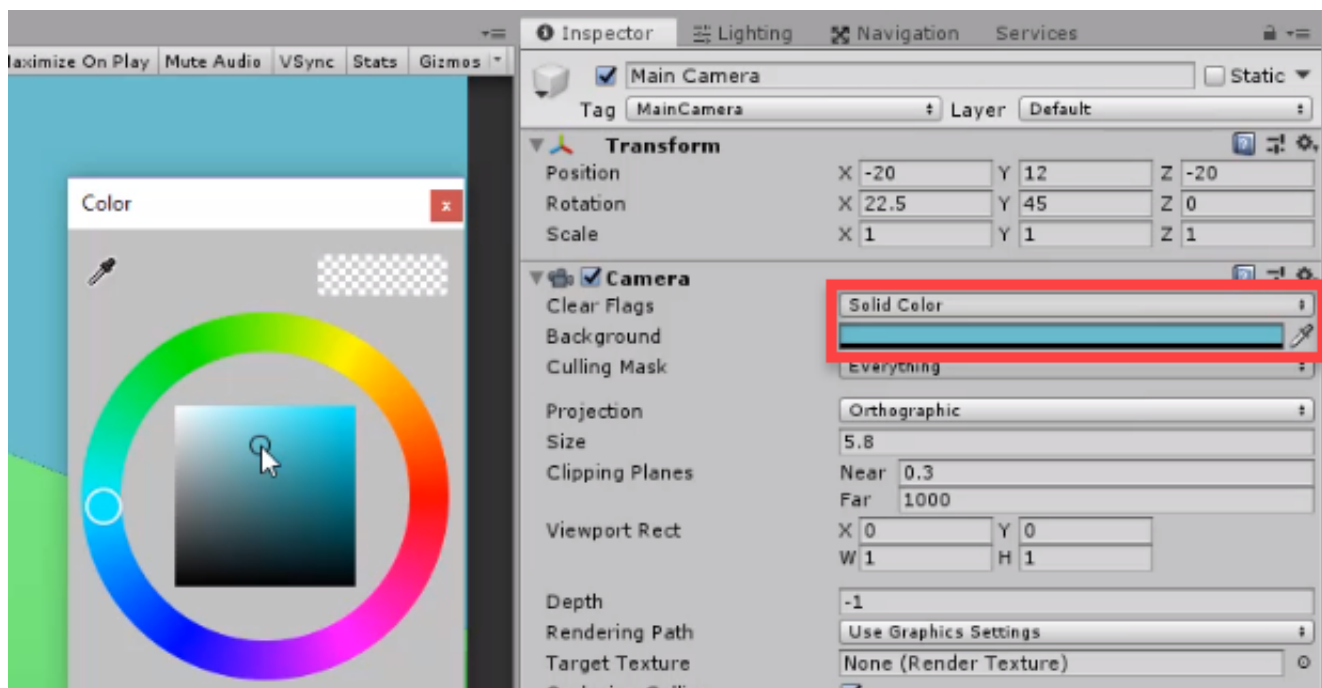
- Set the **Position** to *-20, 12, -20*
- Set the **Rotation** to *22.5, 45, 0*
- Set the **Projection** to *Orthographic*
- Set the **Size** to *5.8*

You might notice that the shadows are quite dark. To fix this, open the **Lighting** window (*Window > Rendering > Lighting*) and click on the **Generate Lighting** button at the bottom of the window.

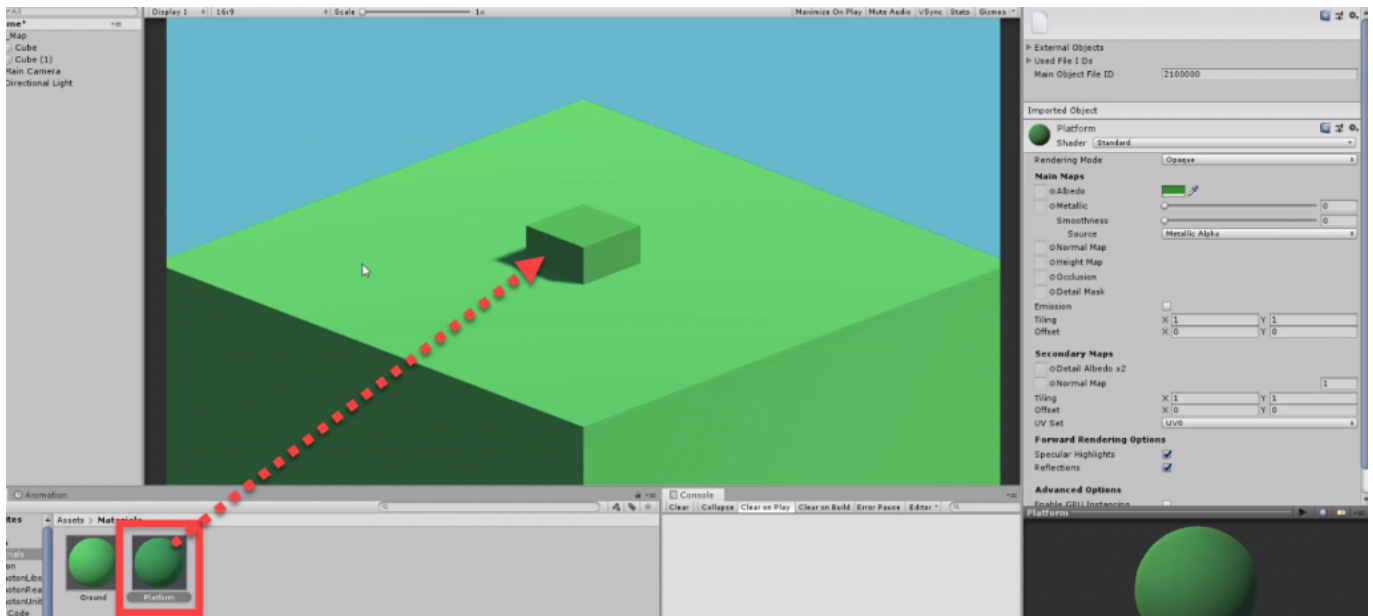


Let's now change the background. Select the camera.

- Set **Clear Flags** to *Solid Color*
- Set **Background** to a light blue

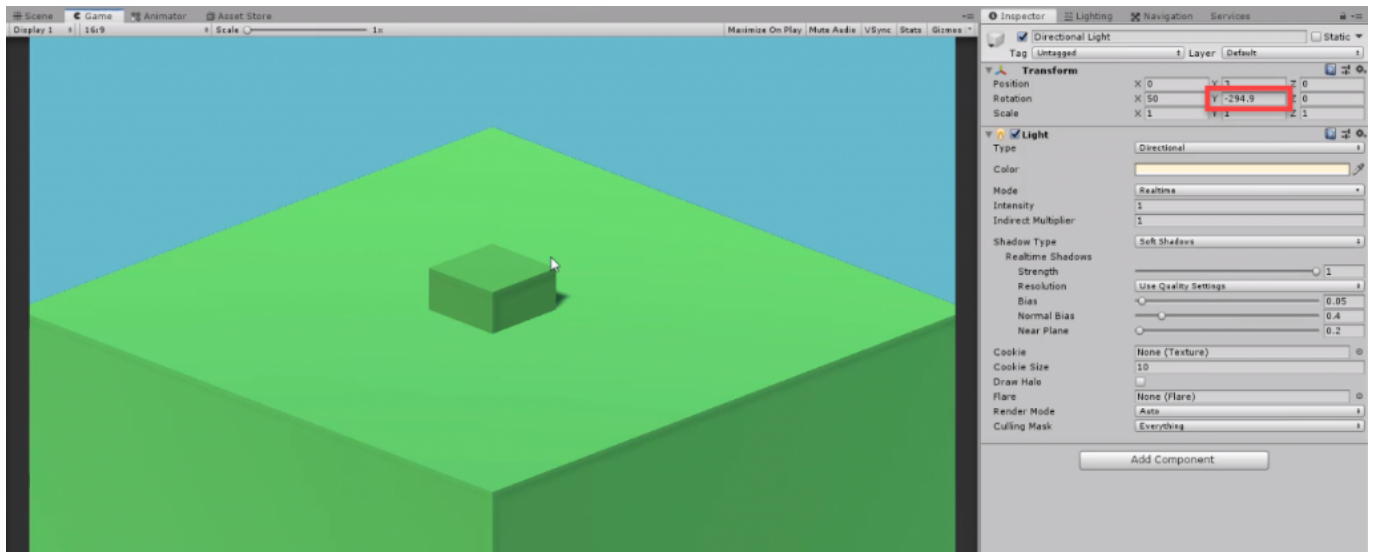


Our middle platform blends in too well with the large one, so let's create a new material to make it stand out.



We can also rotate the light so the shadows are facing behind the platform.

- Set the **Y Rotation** to -290





Network Manager

The network manager will be a custom script that manages connecting to the network.

- Connecting to the master server
- Creating rooms
- Joining rooms
- Changing scenes

In our **Menu** scene, create a new empty GameObject called **_NetworkManager**. This is going to hold our network manager script. Then create a new C# script called **NetworkManager** and attach it to the object.

We have Photon installed, but are unable to use it right now. That's because we need to add it to our using namespaces.

```
using Photon.Pun;
```

Let's also create an instance (singleton) for the script. This basically allows us to access the class from anywhere in the project without needing to find the object, get the component, etc. The only issue is that an instance is only useful for a script with one instance. So an NPC, enemy, etc won't have a singleton.

```
// instance
public static NetworkManager instance;

void Awake ()
{
    // if an instance already exists and it's not this one - destroy us
    if(instance != null && instance != this)
        gameObject.SetActive(false);
    else
    {
        // set the instance
        instance = this;
        DontDestroyOnLoad(gameObject);
    }
}
```

Right now if we launch the game, nothing will happen. We need to connect to Photon.

```
void Start ()
{
    PhotonNetwork.ConnectUsingSettings();
}
```

When we want to create a room, we'll call the **CreateRoom** function, sending over a name for the room.



```
// attempt to create a new room
public void CreateRoom (string roomName)
{
    PhotonNetwork.CreateRoom(roomName);
}
```

When we want to join a room, we'll call the **JoinRoom** function, sending over a name for the room.

```
// attempt to join an existing room
public void CreateRoom (string roomName)
{
    PhotonNetwork.JoinRoom(roomName);
}
```

When we want to change a scene, we're not going to use Unity's default system. Instead we're going to use Photon's and this is because they have some added features to stop sending messages between scene changes and other features with multiplayer in mind.

```
public void ChangeScene (string sceneName)
{
    PhotonNetwork.LoadLevel(sceneName);
}
```



Callback Functions

Right now if you press play in the editor, you'll see that nothing happens. We don't know when we connect to the master server. In order to have some sort of feedback, we need to call a callback function.

First up, let's change the inheriting class from **MonoBehaviour** to **MonoBehaviourPunCallbacks**.

```
public class NetworkManager : MonoBehaviourPunCallbacks
```

So how do we know when we connect to the master server? We need to override the **OnConnectedToMaster** function. This is a function located in the **MonoBehaviourPunCallbacks** class that we can override.

```
public override void OnConnectedToMaster ()
{
    Debug.Log("Connected to master server");
}
```

Now if we press play, we should see after a couple of seconds that we've connected to the master server. When we do so, we can then create a new room and make another callback for when we do so.

```
public override void OnConnectedToMaster ()
{
    CreateRoom("testroom");
}

public override void OnCreatedRoom ()
{
    Debug.Log("Created room: " + PhotonNetwork.CurrentRoom.Name);
}
```

If you press play in the editor now, you should get a console log when you connect to a room.



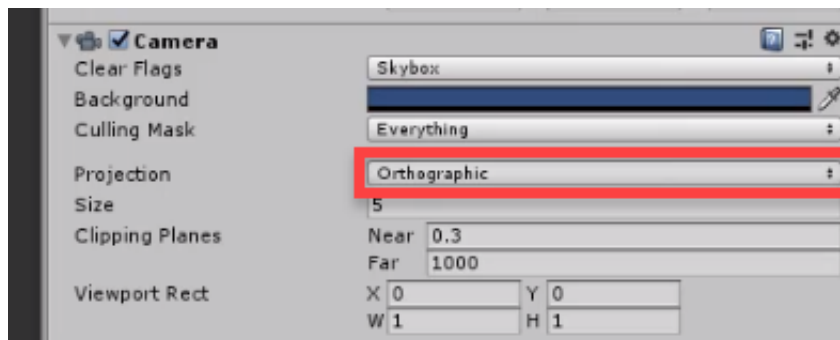
Menu

We're going to have a menu with 2 screens. The main screen will allow the player to enter in a name and room name. Then either choose to create or join the room. When they connect to the room, they will be taken to the lobby screen. Here, all the players in the lobby will be displayed. The host will be able to start the game.



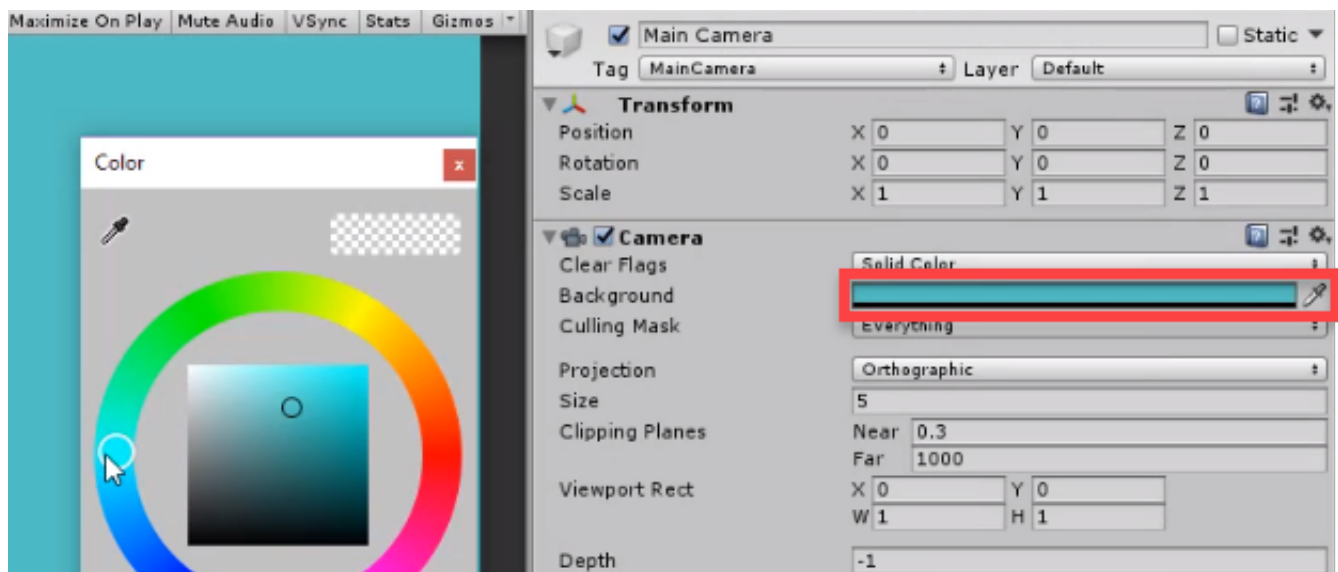
Menu Camera

Inside of our **Menu** scene, let's select the camera and change the **Projection** to *Orthographic* since we don't need depth in our camera.



Let's also change the background color.

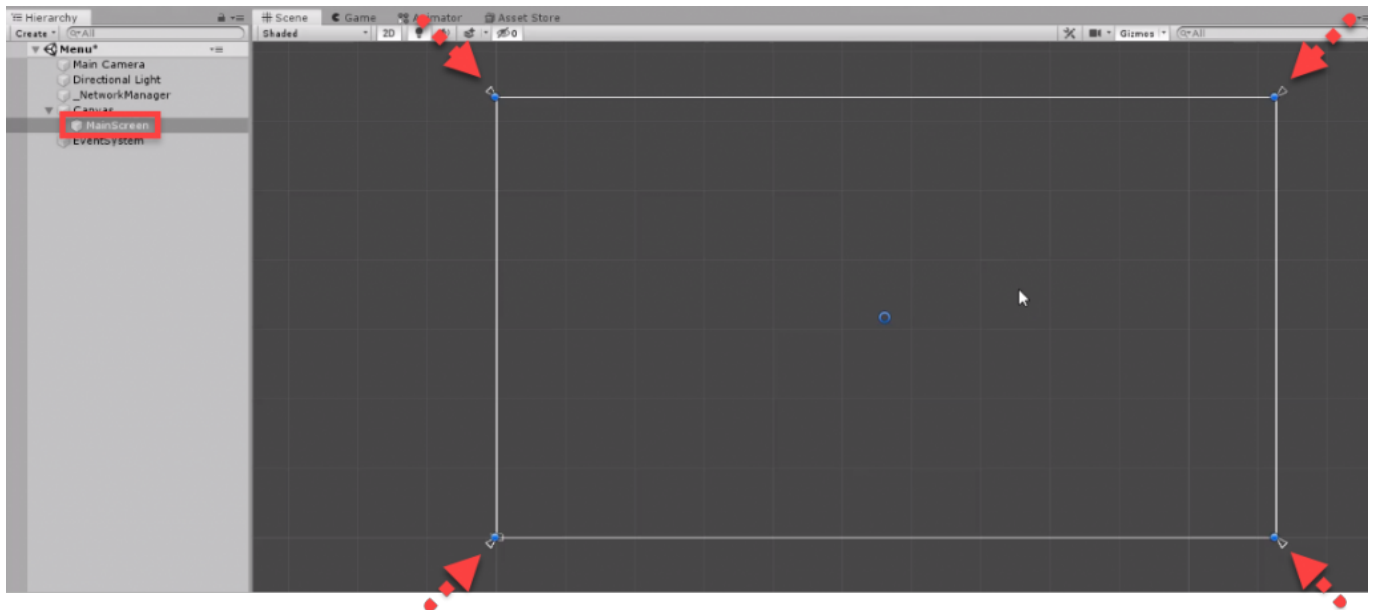
- Set **Clear Flags** to *Solid Color*
- Set **Background** to light blue



Menu UI

Create a new canvas (right click Hierarchy > *UI > Canvas*).

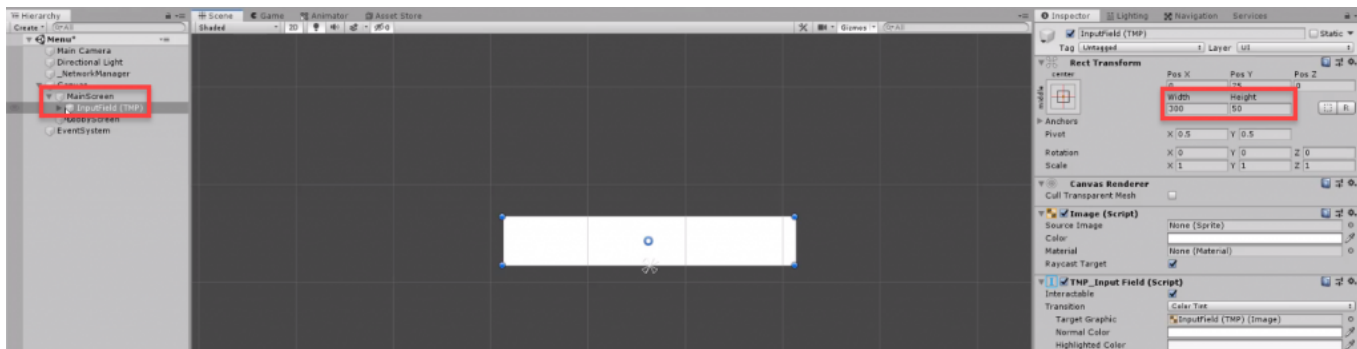
Then create a new empty GameObject (right click Hierarchy > *Create Empty*) and call it **MainScreen**. Resize the rect to make it fill all of the canvas and set the **Anchoring** to *stretch-stretch*.



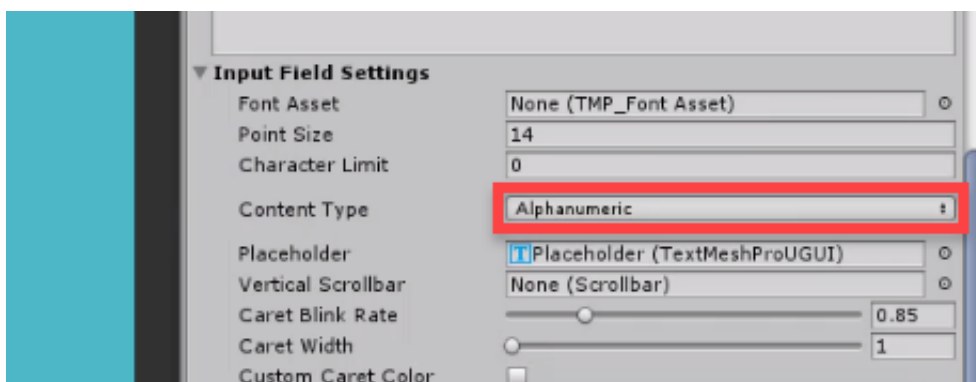
Duplicate the screen and rename it to **LobbyScreen**. These are our 2 screen containers will will contain the UI elements for each.

Now let's create an input field for the main screen. Right click **MainScreen** and select *UI > Input Field - Text Mesh Pro*.

- Set the **Width** to 300
- Set the **Height** to 50

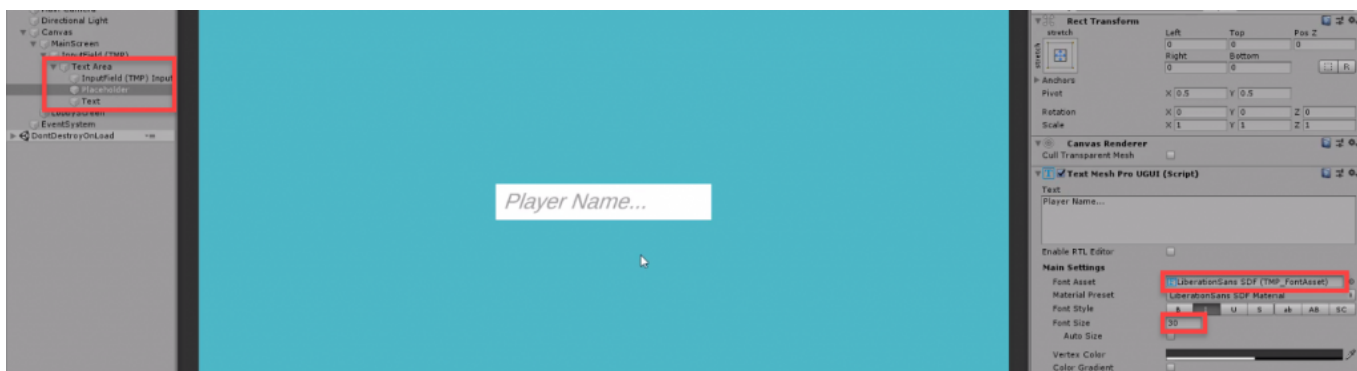


Since we only want the user to enter in numbers and letters, let's set the **Content Type** to *Alphanumeric*.



Then on the placeholder child text:

- Set the **Text** to *Player Name...*

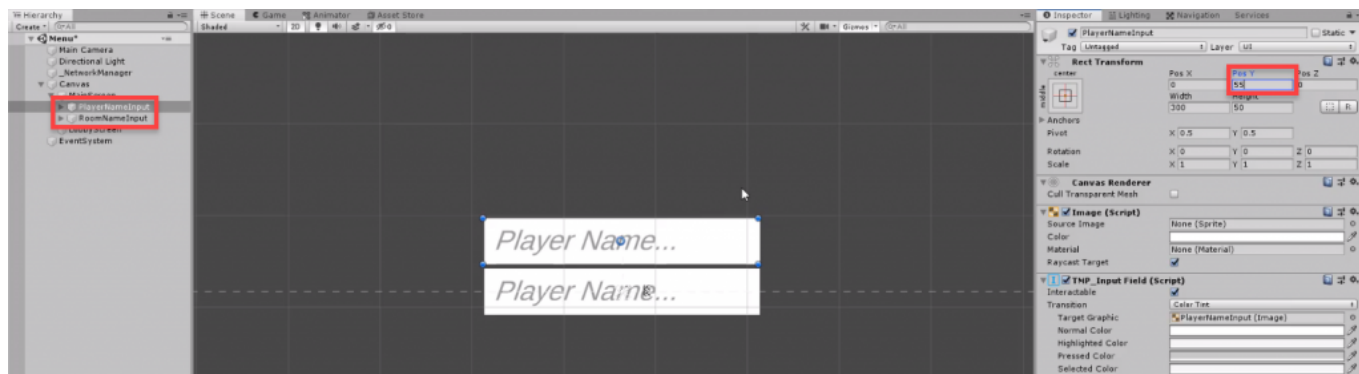


The input field should now be working when you press play.

Main Screen

Duplicate the **PlayerNameInput** button and call it **RoomNameInput**.

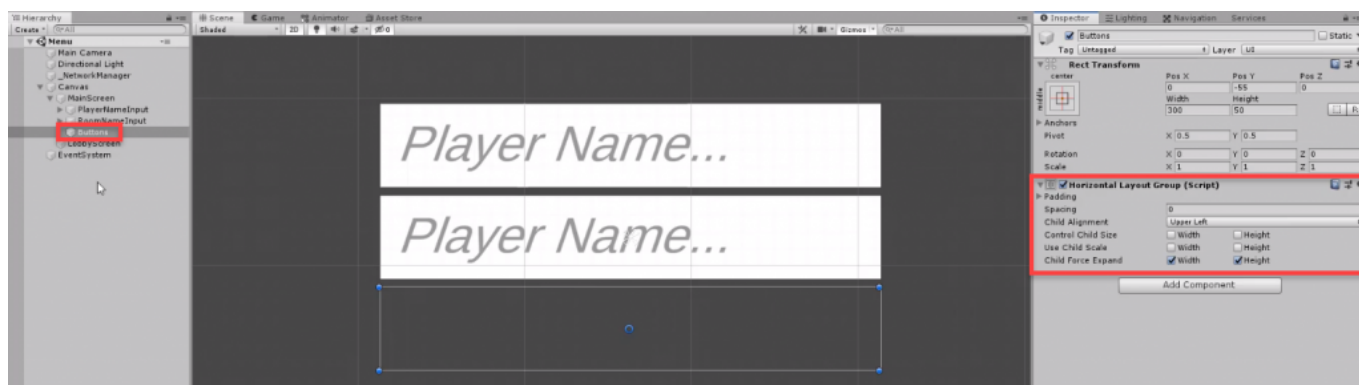
- Set the **Y Position** of **PlayerNameInput** to 55
- Set the **Placeholder** text of **RoomNameInput** to *Room Name...* (not seen in image)



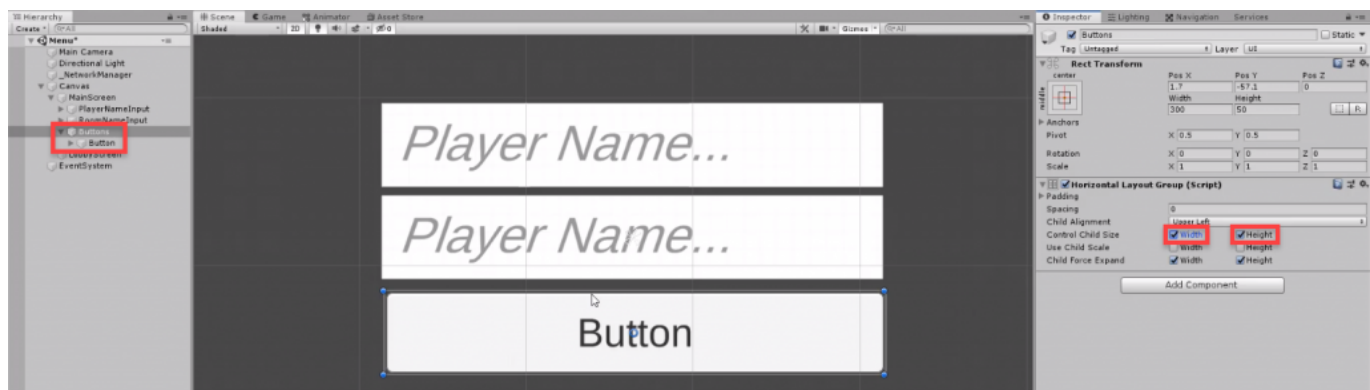
For the two buttons, we're going to use a layout group to evenly and automatically arrange the buttons. Create a new empty object called **Buttons**.

- Set the **Y Position** to -55
- Set the **Width** to 300
- Set the **Height** to 50

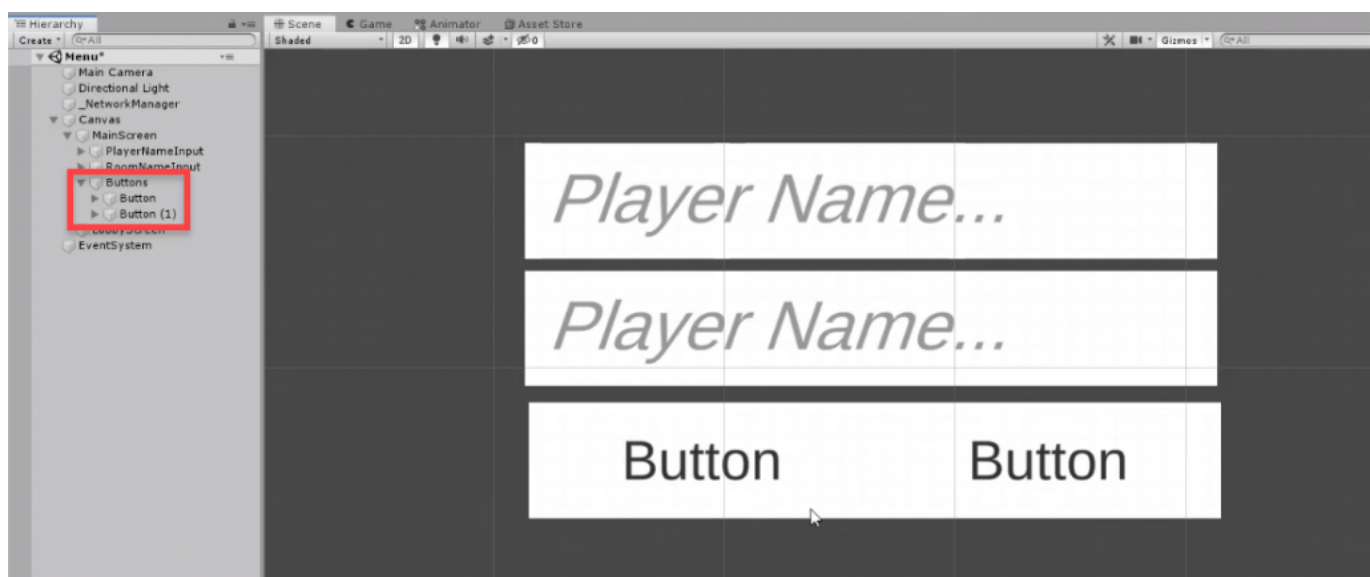
Add a **Horizontal Layout Group** component. This will position and scale child objects automatically and uni-formally.



As a child of that, create a new button (right click Buttons > UI > Button - Text Mesh Pro). To make the layout group component work the way we want, enable **Control Child Size Width** and **Height**.



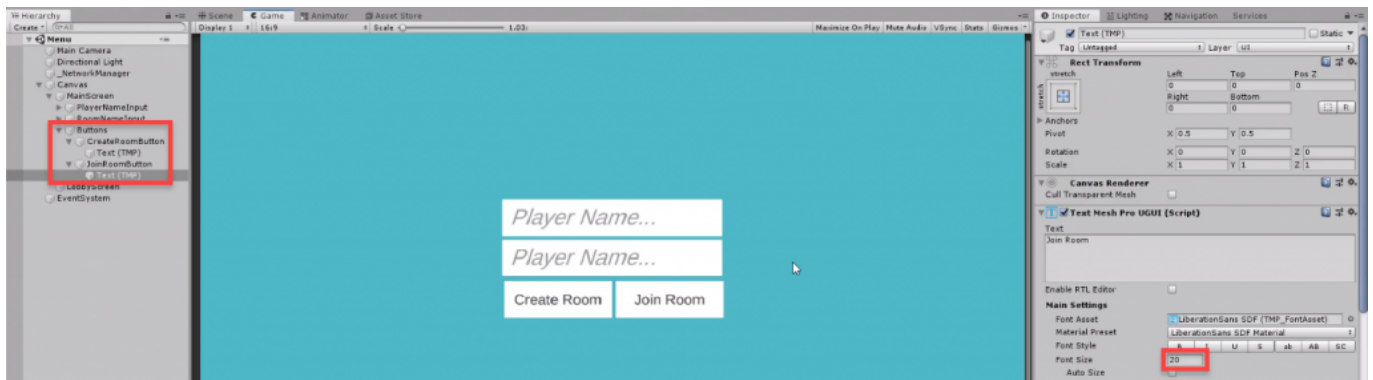
Change the button's source image to *none* then duplicate the button. You should now see that the two buttons are automatically laid out like below.



A problem here though, is that the buttons are right next to each other. To fix this, we can set the layout group component's **Spacing** to 5.

We can now rename the buttons and change the text to match their purpose.

- Set the **Font Size** to 20



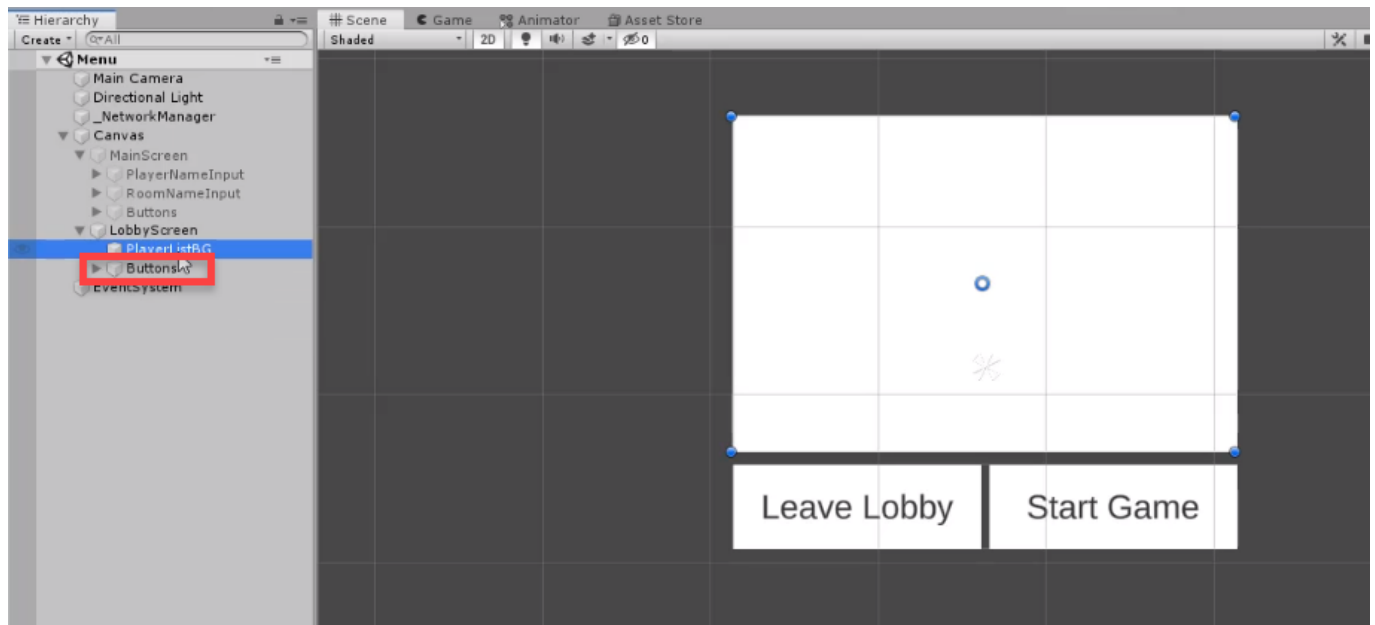
Lobby Screen

To build the lobby screen, let's disable the **MainScreen** object.

As a child of the **LobbyScreen**, create a new Image object called **PlayerListBG**.

- Set the **Y Position** to 50
- Set the **Width** to 300
- Set the **Height** to 200

For our buttons, let's just copy and paste the ones from the main screen and change the buttons text to **Leave Lobby** and **Start Game**.



As a child of the player list background, create a new **Text - Text Mesh Pro** object called **PlayerList**.

- Set the **Anchoring** to *stretch-stretch*
- Set the **Font Size** to 30
- Set the **Vertex Color** to black
- Set the **Alignment** to *center-middle*

Now all of our menu UI is setup and ready to go.



Remote Procedure Calls (RPC)

RPC's allow you to call functions on other player's computers. You can send this to either a specific player or a group of targeted players.

- `PhotonView.RPC("FunctionName", Player, parameters);`
- `PhotonView.RPC("FunctionName", RpcTarget, parameters);`

RpcTarget's are who and how.

- **All** - send to all players in room and call it instantly on your computer
- **AllViaServer** - All, but doesn't call instantly on your computer
- **Others** - send to all players in room but not you
- **MasterClient** - send to the master client
- **AllBuffered** - All, but cache the call for new people who join the room
- **AllBufferedViaServer** - AllViaServer, but cache the call for new people who join the room
- **OthersBuffered** - Others, but cache the call for new people who join the room

Photon View

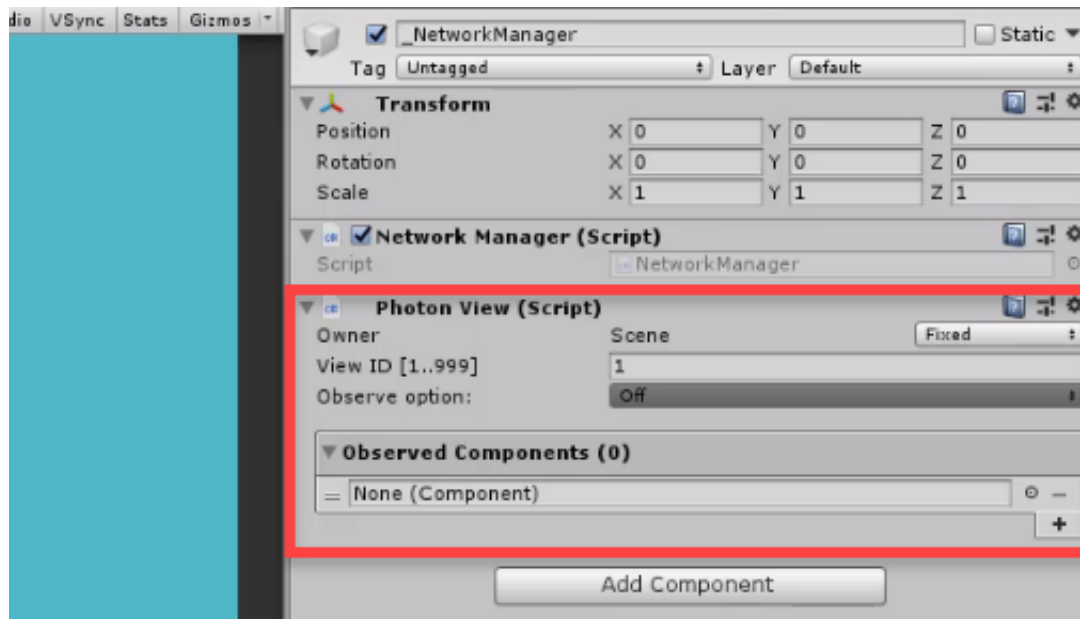
A PhotonView is a component which identifies an object across the network. Each PhotonView has a **View ID** which identifies it across all players computers.

In Unity

In Unity, let's start by tagging our NetworkManager **ChangeScene** function as one that will be used as an RPC. This is because when the host starts the game, they will tell everyone else in the room to call this function. In order to do that, we need to add the **[PunRPC]** attribute.

```
// changes the scene using Photon's system
[PunRPC]
public void ChangeScene (string sceneName)
{
    PhotonNetwork.LoadLevel(sceneName);
}
```

If you have a script which has RPC calls, the object it's attached to needs a PhotonView component. So let's add one to our **_NetworkManager** object.





Menu Script

Let's start scripting our menu, connecting the screens and joining / creating rooms. Create a new object called **_Menu**. Create a new C# script called **Menu** and attach it to the object.

We'll start by adding in our namespaces.

```
using UnityEngine.UI;
using TMPro;
using Photon.Pun;
using Photon.Realtime;
```

For our variables, let's start by referencing each screen.

```
[Header("Screens")]
public GameObject mainScreen;
public GameObject lobbyScreen;
```

Then we can create some variables referencing the things we need on the main screen.

```
[Header("Main Screen")]
public Button createRoomButton;
public Button joinRoomButton;
```

Then for the lobby screen.

```
[Header("Lobby Screen")]
public TextMeshProUGUI playerListText;
public Button startGameButton;
```

In the **Start** function, let's deactivate the main screen buttons since we don't want the player to be able to do network stuff when we're not connected to the master server.

```
void Start ()
{
    // disable the buttons at the start as we're not connected to the server yet
    createRoomButton.interactable = false;
    joinRoomButton.interactable = false;
}
```

In order to get Photon callbacks, let replace the inheriting class.

```
public class Menu : MonoBehaviourPunCallbacks
```

Then when we connect to the master server, enable the buttons.

```
// called when we connect to the master server
// enables the "Create Room" and "Join Room" buttons
public override void OnConnectedToMaster ()
{
    createRoomButton.interactable = true;
    joinRoomButton.interactable = true;
}
```

Let's go back to Unity and drag in all the properties. If you press play, the main screen buttons should be disabled until we connect to the master server.

More Functions

Let's now create the **SetScreen** function.

```
void SetScreen (GameObject screen)
{
    // deactivate all screens
    mainScreen.SetActive(false);
    lobbyScreen.SetActive(false);

    // enable the requested screen
    screen.SetActive(true);
}
```

When we click on a button, we're going to call a specific function in the script. Let's create the **OnCreateRoomButton** function, which gets called when we click on the **Create Room** button.

```
public void OnCreateRoomButton (TMP_InputField roomNameInput)
{
    NetworkManager.instance.CreateRoom(roomNameInput.text);
}
```

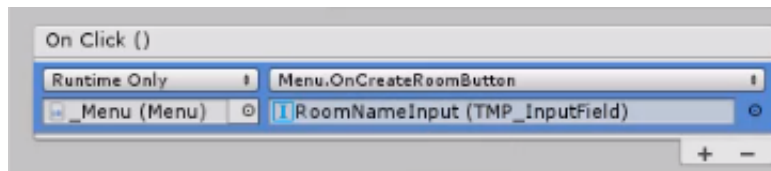
And then for when we click on the **Join Room** button.

```
public void OnJoinRoomButton (TMP_InputField roomNameInput)
{
    NetworkManager.instance.JoinRoom(roomNameInput.text);
}
```

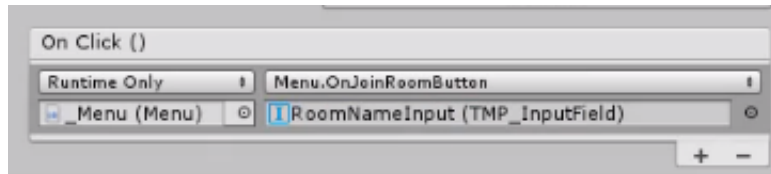
Connecting the Buttons

Back in the editor, let's connect the main screen buttons to the script.

Create Room button.



Join Room button.

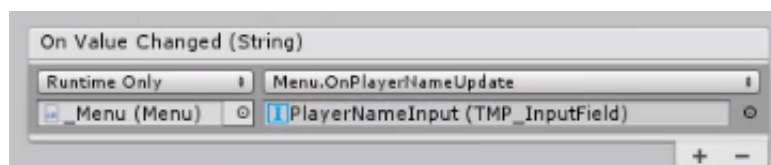


Updating the Player Name

Let's now make the player name update when we fill in the input field.

```
public void OnPlayerNameUpdate (TMP_InputField playerNameInput)
{
    PhotonNetwork.NickName = playerNameInput.text;
}
```

In the editor, select the player name input and scroll down to the **On Value Changed** event. This will call the function whenever the player types something in the input field.



Lobby

When we create or join a room, we'll check for the **OnJoinedRoom** callback and then switch over to the lobby screen.

```
public override void OnJoinedRoom ()
{
    SetScreen(lobbyScreen);
}
```

Then we're going to create the **UpdateLobbyUI** function, which gets called whenever someone joins or leaves the lobby. It will also be tagged as an RPC because we'll be calling this across the network.

```
[PunRPC]
public void UpdateLobbyUI ()
{
    playerListText.text = "";

    // display all the players currently in the lobby
    foreach(Player player in PhotonNetwork.PlayerList)
    {
        playerListText.text += player.NickName + "\n";
    }

    // only the host can start the game
    if(PhotonNetwork.IsMasterClient)
        startGameButton.interactable = true;
    else
        startGameButton.interactable = false;
}
```

Let's go back to the **OnJoinedRoom** function and call the **UpdateLobbyUI** as an RPC to all players in the room.

```
// since there's now a player in the lobby, tell everyone to update the lobby UI
photonView.RPC("UpdateLobbyUI", RpcTargets.All);
```

Then we need to update the lobby UI when a player leaves the lobby. This is done through another callback function called **OnPlayerLeftRoom**.

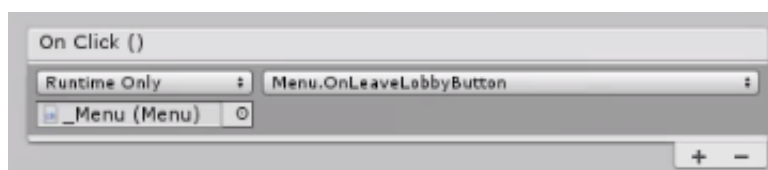
```
public override void OnPlayerLeftRoom (Player otherPlayer)
{
    // we don't RPC it like when we join the lobby
    // that's because OnJoinRoom is only called for the client who just joined
    // OnPlayerLeftRoom gets called for all clients in the room, so we don't need to
    RPC
    UpdateLobbyUI();
}
```

Let's continue with the lobby buttons now.

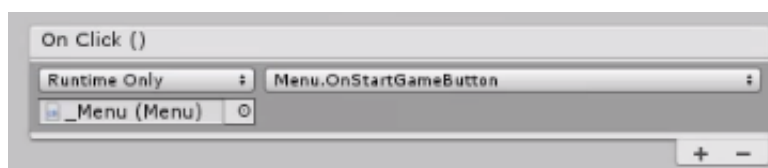
```
public void OnLeaveLobbyButton ()
{
    PhotonNetwork.LeaveRoom();
    SetScreen(mainScreen);
}

public void OnStartGameButton ()
{
    NetworkManager.instance.photonView.RPC("ChangeScene", RpcTargets.All, "Game");
}
```

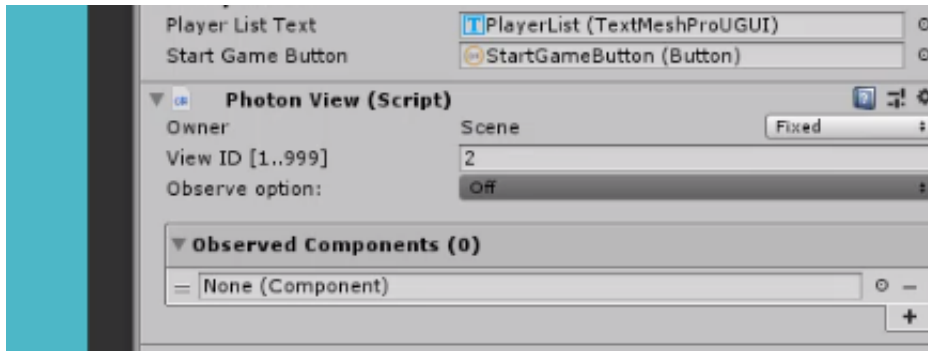
Leave Lobby button.



Start Game button.



Since we're calling RPC's in the **Menu** script, we'll need to add a PhotonView component to the **_Menu** object.



You should now be able to create and join rooms, enter the lobby and start the game with a group of players.



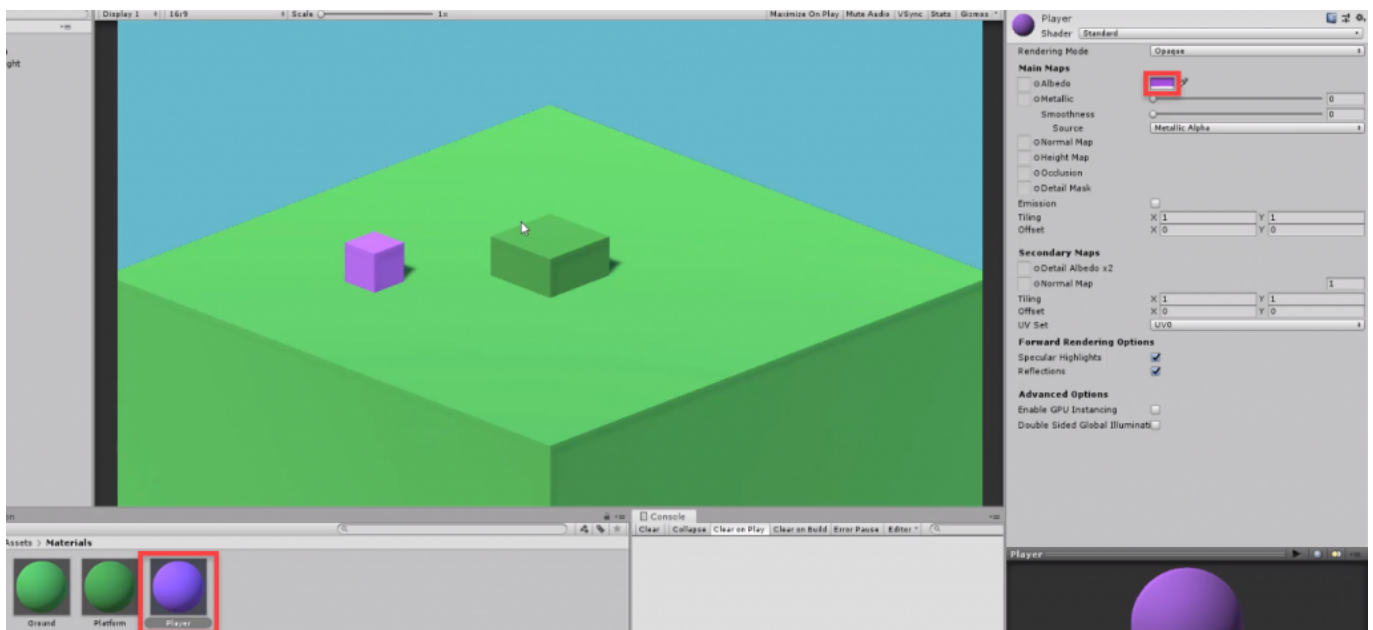
Player Controller

We're going to create a player controller which can move, jump and steal the hat from other players by hitting them. When the player holds the hat, their timer increases. If it reaches the max – they win.

Create a new cube object and call it **Player**. Then set the tag as **Player**.

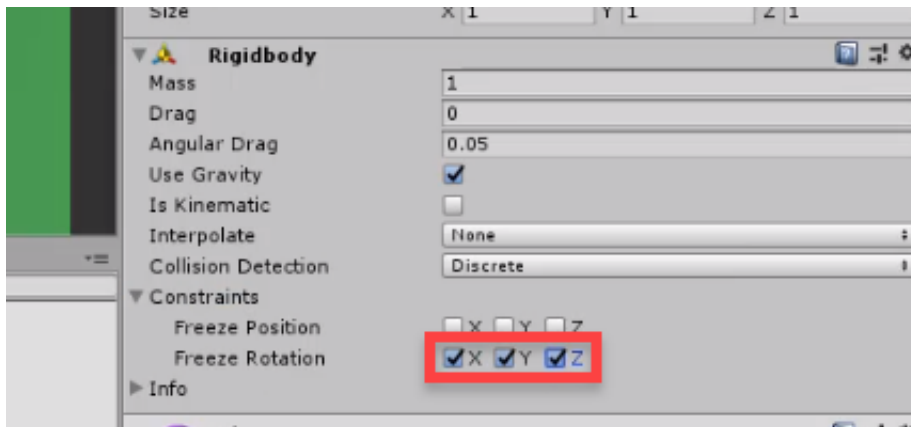
Now create a new Material called **Player** and apply it.

- Set the **Albedo** color to purple
- Set the **Smoothness** to 0



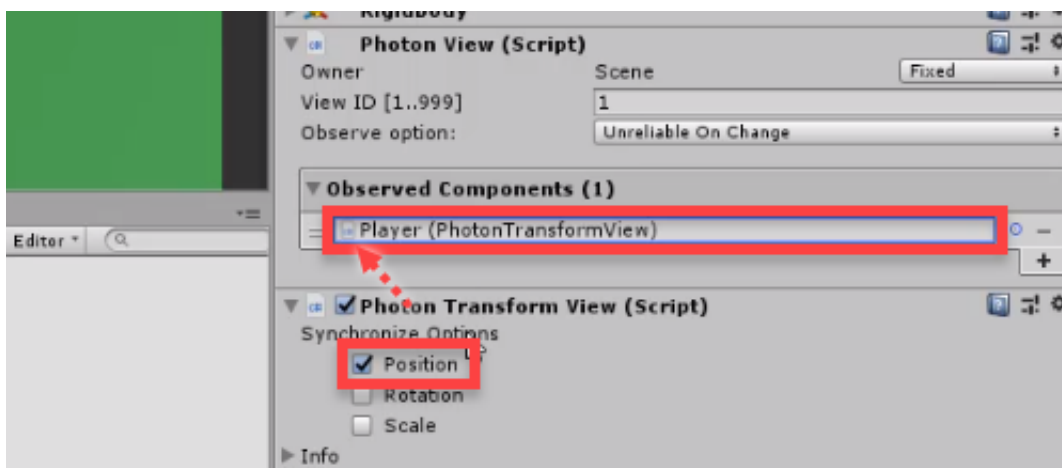
Now add a **Rigidbody** component.

- Enable **Freeze Rotation** x, y, z

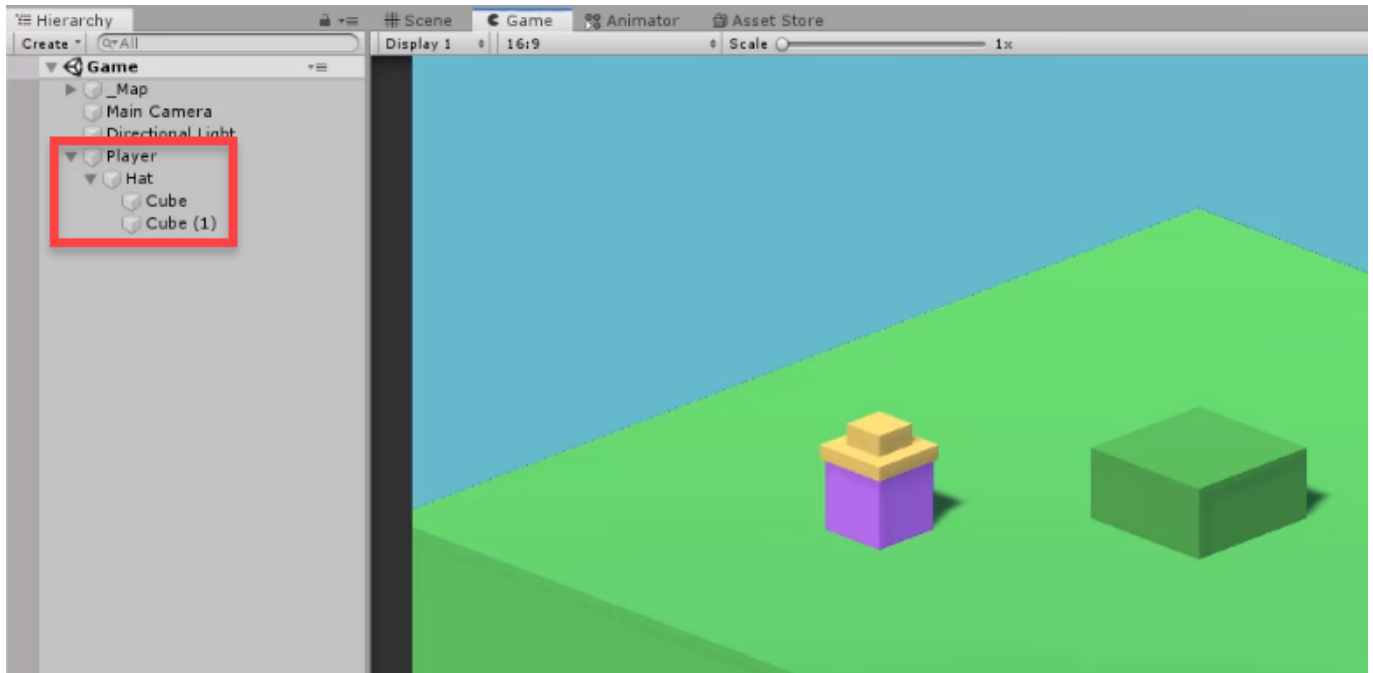


Then add both a **PhotonView** and **PhotonTransformView**. A PhotonTransformView syncs the object's transform component across the network to all other players. This is a quick way to do this and doesn't require us to write any syncing code in a script.

- Drag the **PhotonTransformView** component into the **PhotonView's Observed Components** list
- Set **Synchronize Options** to *Position*



Let's now create the hat object. This can be whatever you wish.



PlayerController Script

Create a new C# script called **PlayerController** and attach it to the player object.

Let's start by adding in our namespaces.

```
using Photon.Pun;  
using Photon.Realtime;
```

Then we can start with our variables.

```
[HideInInspector]  
public int id;  
  
[Header("Info")]  
public float moveSpeed;  
public float jumpForce;  
public GameObject hatObject;  
  
[HideInInspector]  
public float curHatTime;  
  
[Header("Components")]  
public Rigidbody rig;  
public Player photonPlayer;
```



PlayerController Script

In the **Update** function, we'll call the move and jump functions.

```
void Update ()
{
    Move();

    if(Input.GetKeyDown(KeyCode.Space))
        TryJump();
}
```

The **Move** function, checks for keyboard input and sets the rigidbody's velocity.

```
void Move ()
{
    float x = Input.GetAxis("Horizontal") * moveSpeed;
    float z = Input.GetAxis("Vertical") * moveSpeed;

    rig.velocity = new Vector3(x, rig.velocity.y, z);
}
```

The **TryJump** function checks if we're standing on the ground and if so, add force upwards.

```
void TryJump ()
{
    Ray ray = new Ray(transform.position, Vector3.down);

    if(Physics.Raycast(ray, 0.7f))
        rig.AddForce(Vector3.up * jumpForce, ForceMode.Impulse);
}
```

Back in the editor, we can fill in the properties and try entering in some values which feel good to play.



GameManager Script

Create a new C# script called **GameManager** and attach it to a new object called **_GameManager**.

Let's add the namespaces.

```
using Photon.Pun;  
using Photon.Realtime;  
using System.Linq;
```

We then need to change the inherited class.

```
public class GameManager : MonoBehaviourPunCallbacks
```

Here's the variables.

```
[Header("Stats")]  
public bool gameEnded = false;           // has the game ended?  
public float timeToWin;                   // time a player needs to hold the hat to win  
public float invincibleDuration;          // how long after a player gets the hat, are th  
ey invincible  
private float hatPickupTime;              // the time the hat was picked up by the curren  
t holder  
  
[Header("Players")]  
public string playerPrefabLocation;        // path in Resources folder to the Player prefa  
b  
public Transform[] spawnPoints;           // array of all available spawn points  
public PlayerController[] players;         // array of all the players  
public int playerWithHat;                  // id of the player with the hat  
private int playersInGame;                 // number of players in the game  
  
// instance  
public static GameManager instance;  
  
void Awake ()  
{  
    // instance  
    instance = this;  
}
```

In the **Start** function, let's setup the players array.

```
void Start ()  
{  
    players = new PlayerController[PhotonNetwork.PlayerList.Length];  
    photonView.RPC("ImInGame", RpcTarget.All);  
}
```



The **ImInGame** function gets called when a player joins the game.

```
[PunRPC]
void ImInGame ()
{
    playersInGame++;

    if(playersInGame == PhotonNetwork.PlayerList.Length)
        SpawnPlayer();
}
```

The **SpawnPlayer** function gets called when all the players are in the game.

```
// spawns a player and initializes it
void SpawnPlayer ()
{
    // instantiate the player across the network
    GameObject playerObj = PhotonNetwork.Instantiate(playerPrefabLocation, spawnPoints[Random.Range(0, spawnPoints.Length)], Quaternion.Identity);

    // get the player script
    PlayerController playerScript = playerObj.GetComponent<PlayerController>();
}
```



Initializing the Player

In the **PlayerController** script the **Initialize** function will be called which sets up the player object.

```
// called when the player object is instantiated
[PunRPC]
public void Initialize (Player player)
{
    photonPlayer = player;
    id = player.ActorNumber;

    GameManager.instance.players[id - 1] = this;

    // give the first player the hat

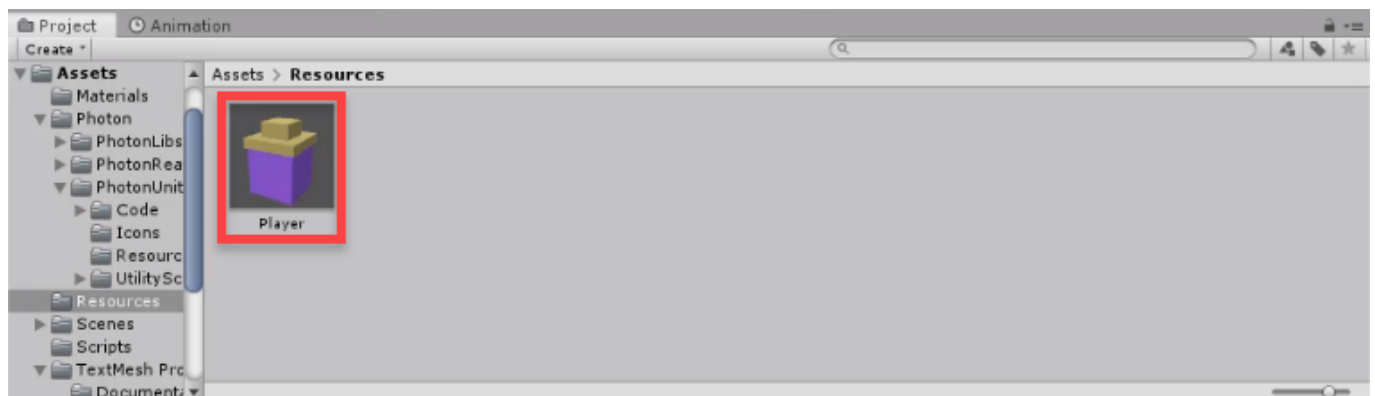
    // if this isn't our local player, disable physics as that's
    // controlled by the user and synced to all other clients
    if(photonView.isMine)
        rig.isKinematic = true;
}
```

Back in the **GameManager** script, let's initialize the player in the **SpawnPlayers** function.

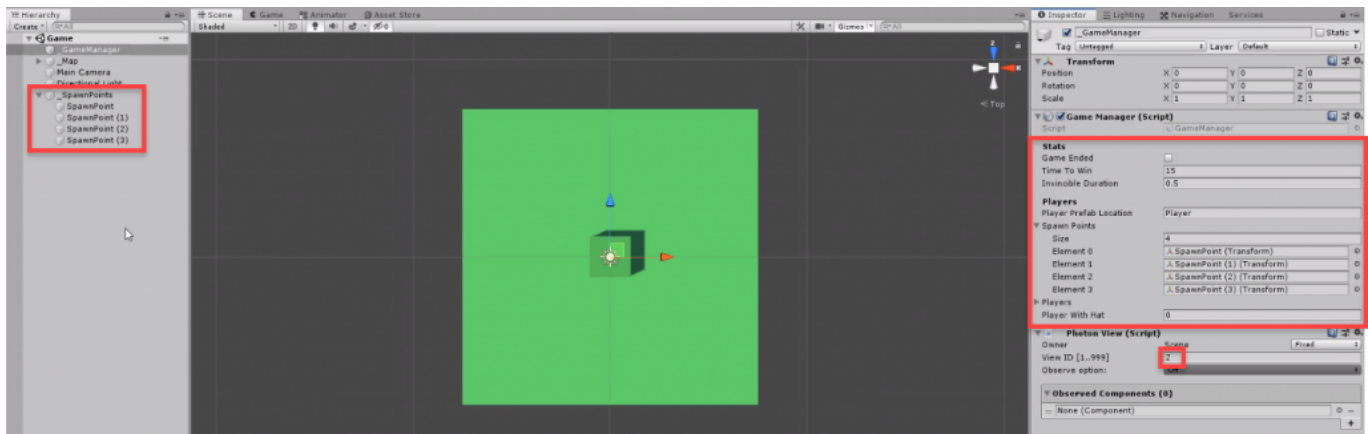
```
// initialize the player
playerScript.photonView.RPC("Initialize", RpcTargets.All, PhotonNetwork.LocalPlayer);
```

Back in the Editor

Back in the Editor, let's begin by saving the player object as a prefab in the **Resources** folder.



- Create a number of different spawn points (empty GameObjects)
- Set **Time to Win** to 15
- Set **Invincible Duration** to 0.5
- Set **Player Prefab Location** to *Player*
- Add the spawn points to the **Spawn Points** array
- Set the PhotonView's **ViewID** to 2



You can now build the game and test it out with multiple instances.

GameManager Script

Back in the **GameManager** script, let's create two functions that take in an id or game object, then returns the corresponding player.

```
public PlayerController GetPlayer (int playerId)
{
    return players.First(x => x.id == playerId);
}

public PlayerController GetPlayer (GameObject playerObject)
{
    return players.First(x => x.gameObject == playerObject);
}
```



Giving the Hat Between Players

In the **PlayerController** script, we're going to create the **SetHat** function, which either enables or disables the hat visual.

```
// sets the player's hat active or not
public void SetHat (bool hasHat)
{
    hatObject.SetActive(hasHat);
}
```

In the **GameManager** script, we're going to create the **GiveHat** function. This will be an RPC that everyone receives and swaps the hat visual between the two players.

```
// called when a player hits the hatted player - giving them the hat
[PunRPC]
public void GiveHat (int playerId, bool initialGive)
{
    // remove the hat from the currently hatted player
    if(!initialGive)
        GetPlayer(playerWithHat).SetHat(false);

    // give the hat to the new player
    playerWithHat = playerId;
    GetPlayer(playerId).SetHat(true);
    hatPickupTime = Time.time;
}
```

We also need a function to check if we can actually get the hat.

```
// is the player able to take the hat at this current time?
public bool CanGetHat ()
{
    if(Time.time > hatPickupTime + invincibleDuration)
        return true;
    else
        return false;
}
```

Back in the **PlayerController** script, let's check for collisions in the **OnCollisionEnter** function.

```
void OnCollisionEnter (Collision collision)
{
    if(!photonView.IsMine)
        return;

    // did we hit another player?
    if(collision.gameObject.CompareTag("Player"))
    {
```




```
// do they have the hat?
if(GameManager.instance.GetPlayer(collision.gameObject).id == GameManager.ins
tance.playerWithHat)
{
    // can we get the hat?
    if(GameManager.instance.CanGetHat())
    {
        // give us the hat
        GameManager.instance.photonView.RPC("GiveHat", RpcTarget.All, id, fal
se);
    }
}
}
```

At the start of the game, we want to give the first player the hat. We'll check and call this in the **Initialize** function.

```
// give the first player the hat
if(id == 1)
    GameManager.instance.GiveHat(id, true);
```

In the editor, let's go to the **Player** prefab and deactivate the hat object so they start without it.

You should now be able to build the project, start up 2 instances and play the game. The hat swapping should also work.



Making a Player Win

We're going to now make it so once a player has held the hat for a certain amount of time, they win, we wait a few seconds then return to the menu scene.

In the **PlayerController**, let's first fix up the **Update** function to make sure only the local player can control their player.

```
if(photonView.isMine)
{
    Move();

    if(Input.GetKeyDown(KeyCode.Space))
        TryJump();

    // track the amount of time we're wearing the hat
    if(hatObject.activeInHierarchy)
        curHatTime += Time.deltaTime;
}
```

Then above that, we can count up the player's current hat time.

```
// the host will check if the player has won
if(PhotonNetwork.IsMasterClient)
{
    if(curHatTime >= GameManager.instance.timeToWin && !GameManager.instance.gameEnded)
    {
        GameManager.instance.gameEnded = true;
        GameManager.instance.photonView.RPC("WinGame", RpcTarget.All, id);
    }
}
```

In the **GameManager** script, let's create the **WinGame** function.

```
[PunRPC]
void WinGame (int playerId)
{
    gameEnded = true;
    PlayerController player = GetPlayer(playerId);
    // set the UI to show who's won

    Invoke("GoBackToMenu", 3.0f);
}
```

The **GoBackToMenu** function gets called 3 seconds after a player wins the game.

```
void GoBackToMenu ()
{
```



```
PhotonNetwork.LeaveRoom();  
NetworkManager.instance.ChangeScene("Menu");  
}
```

Syncing the Player Hat Time

Since we have the master client checking each player's cur hat time and each player needs all the others to show their UI (we'll do this soon), we need to sync our hat time across the network. This could be done by an RPC, calling a function each frame, yet that's quite bulky and unnecessary.

The first thing we need to do is add a second inherited class: **IPunObservable**

```
public class PlayerController : MonoBehaviourPunCallbacks, IPunObservable
```

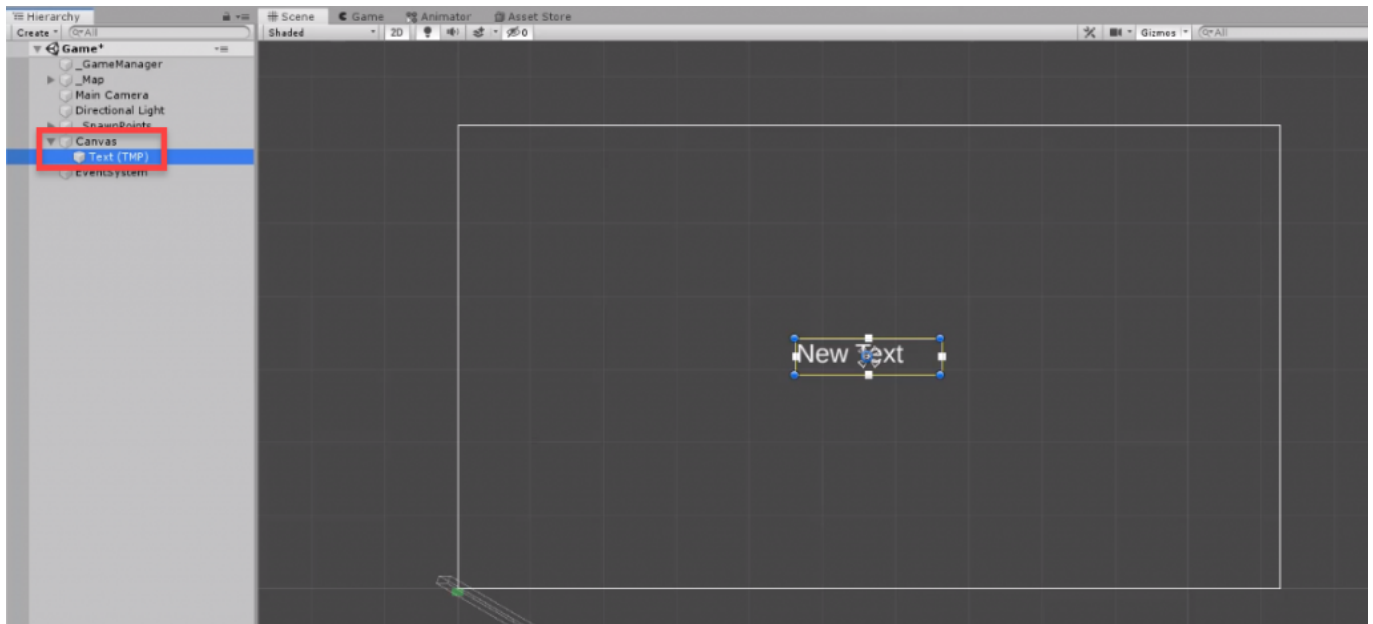
Then we can create the **OnPhotonSerializeView** function, which allows us to send and receive data.

```
public void OnPhotonSerializeView (PhotonStream stream, PhotonMessageInfo info)  
{  
    if(stream.IsWriting)  
    {  
        stream.SendNext(curHatTime);  
    }  
    else if(stream.IsReading)  
    {  
        curHatTime = (float)stream.ReceiveNext();  
    }  
}
```

In the next lesson, we'll create the game UI.

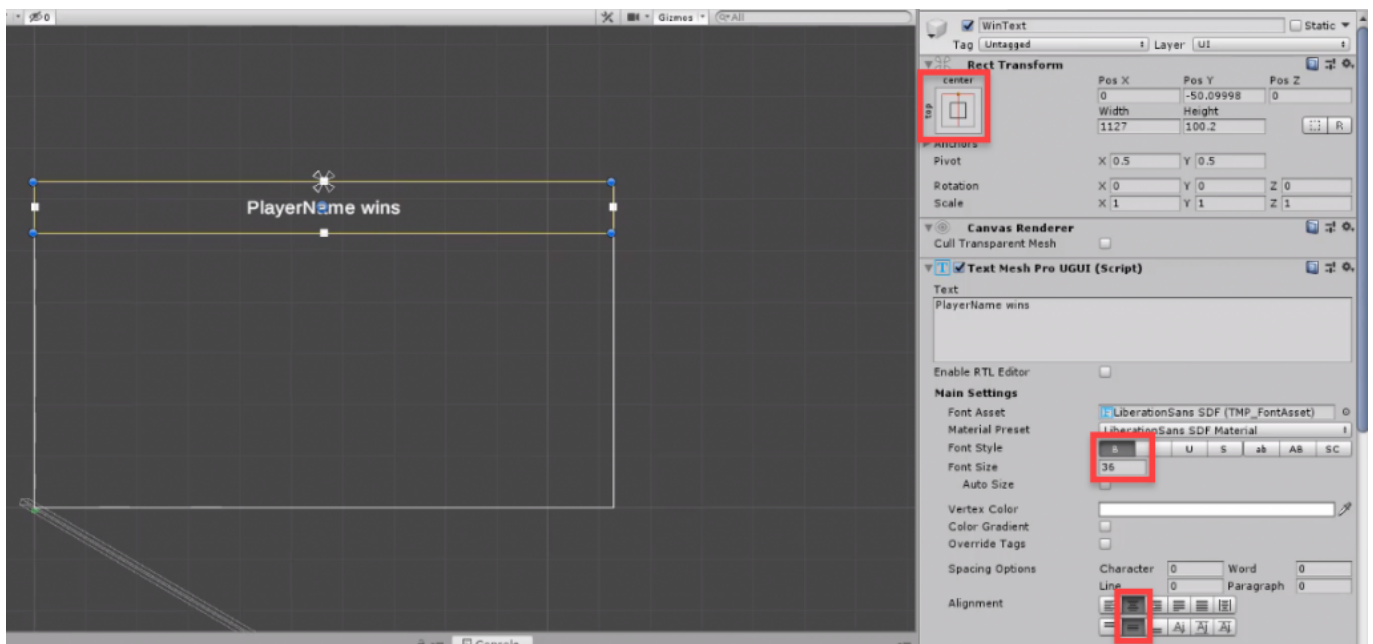
Win Text

First thing, create a new canvas and as a child of that, create a Text - TextMeshPro object. Call it **WinText**.



Move the **WinText** to the top of the screen and stretch it out like so.

- Set the **Anchoring** to *top-center*
- Set the **Font Style** to *Bold*
- Set the **Font Size** to 36
- Set the **Alignment** to *center-middle*



Players List

For the players list, create a new empty object and move it to the top left of the screen. Call

this **Players**.

- Set the **Width** to 200
- Set the **Height** to 350 (flexible)
- Set the **Anchoring** to *top-left*
- Add a **VerticalLayoutGroup** component



As a child of the **Players** container, create a new empty object called **Player**.

- Set the **Width** to 300
- Set the **Height** to 100

As a child of that, create a new text mesh pro text object called **PlayerName**.

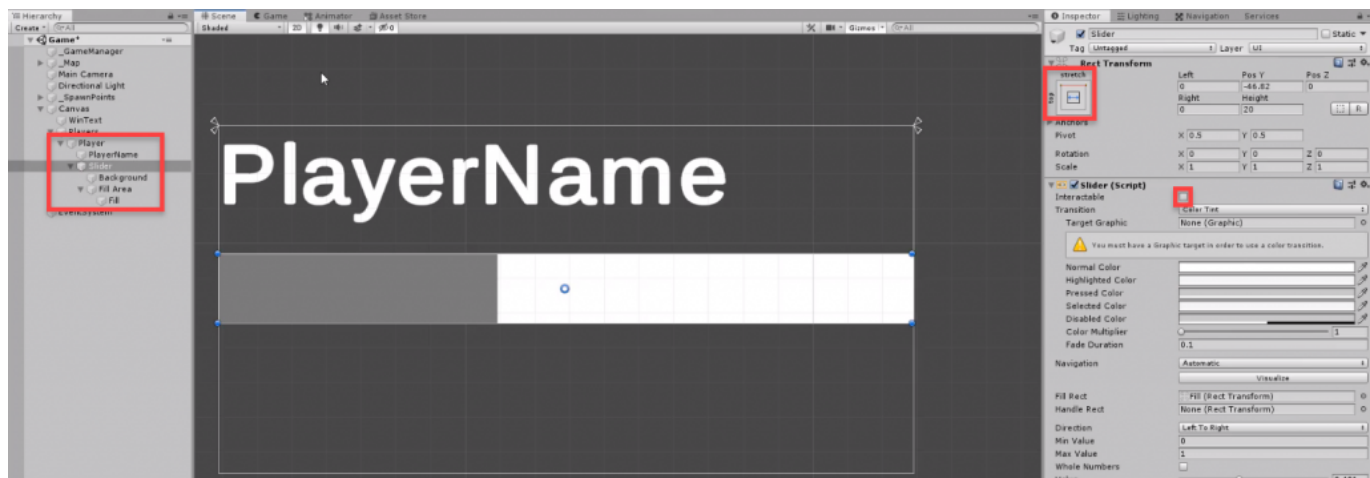
- Set the **Anchoring** to *top-stretch*
- Set the **Height** to 50
- Set the **Font Style** to *Bold*
- Set the **Font Size** to 25
- Set the **Alignment** to *left-top*

For the hat time progress bar, let's create a new slider as a child of the **Player** object.

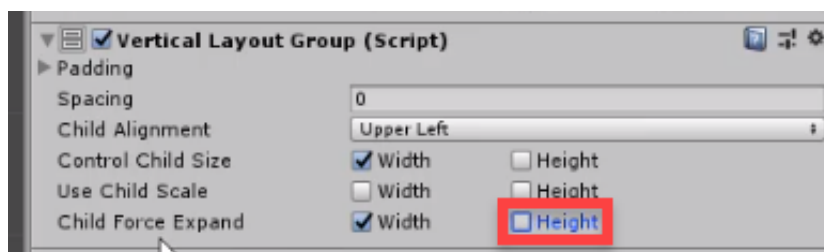
- Set the Background **Source Image** to *None*
- Set **Interactable** to *false*
- Delete the **Handle Slide Area** object



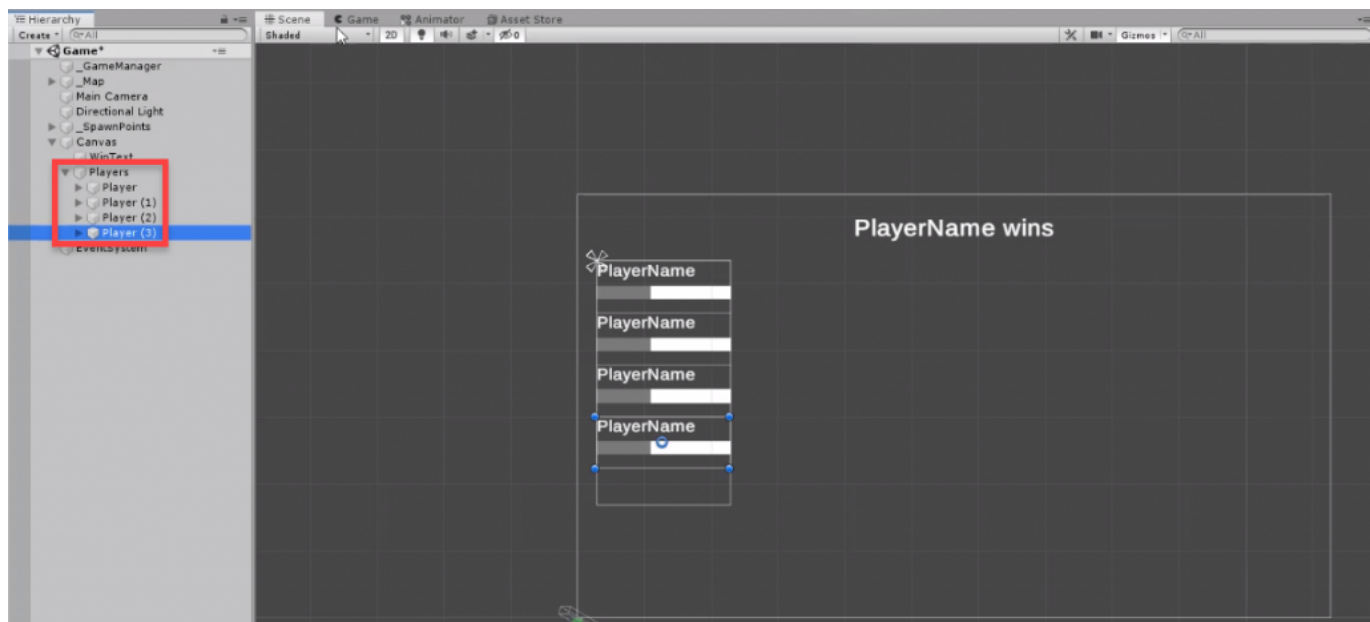
- Set the Fill Area > Fill **Source Image** to **None**
- Set the Fill Area > Fill **Color** to a dark grey



Select the **Players** object and disable **Child Force Expand - Height**. This will make out player containers stack up rather than be spread out evenly.



Now we can duplicate the player containers and they should stack up like below.



After this, disable all of them and we can continue on with the script in the next lesson.



GameUI Script

Create a new C# script called **GameUI** and attach it to the **_GameManager** object.

The first thing we want to do, is add in our namespaces we'll be using.

```
using UnityEngine.UI;
using TMPro;
using Photon.Pun;
```

At the bottom of the script, let's create a new class which will hold all the info for each of those player containers we made last lesson.

```
[System.Serializable]
public class PlayerUIContainer
{
    public GameObject obj;
    public TextMeshProUGUI nameText;
    public Slider hatTimeSlider;
}
```

Back up in our **GameUI** class, we can enter in our variables.

```
public PlayerController[] playerContainers;
public TextMeshProUGUI winText;
```

Now let's also create an instance of the class so we can easily access it.

```
// instance
public static GameUI instance;

void Awake ()
{
    // set the instance to this script
    instance = this;
}
```

The **InitializePlayerUI** function will be called at the start of the game.

```
void Start ()
{
    InitializePlayerUI();
}

void InitializePlayerUI ()
{
    // loop through all containers
```



```
for(int x = 0; x < playerContainers.Length; ++x)
{
    PlayerUIContainer container = playerContainers[x];

    // only enable and modify UI containers we need
    if(x < PhotonNetwork.PlayerList.Length)
    {
        container.obj.SetActive(true);
        container.nameText.text = PhotonNetwork.PlayerList[x].NickName;
        container.hatTimeSlider.maxValue = GameManager.instance.timeToWin;
    }
    else
        container.obj.SetActive(false);
}
```

The **UpdatePlayerUI** function gets called every frame and updates the UI slider bars current value.

```
void Update ()
{
    UpdatePlayerUI();
}

void UpdatePlayerUI ()
{
    // loop through all players
    for(int x = 0; x < GameManager.instance.players.Length; ++x)
    {
        if(GameManager.instance.players[x] != null)
            playerContainers[x].hatTimeSlider.value = GameManager.instance.players[x]
            .curHatTime;
    }
}
```

When the game is finished, the **SetWinText** function will be called.

```
public void SetWinText (string winnerName)
{
    winText.gameObject.SetActive(true);
    winText.text = winnerName + " wins";
}
```

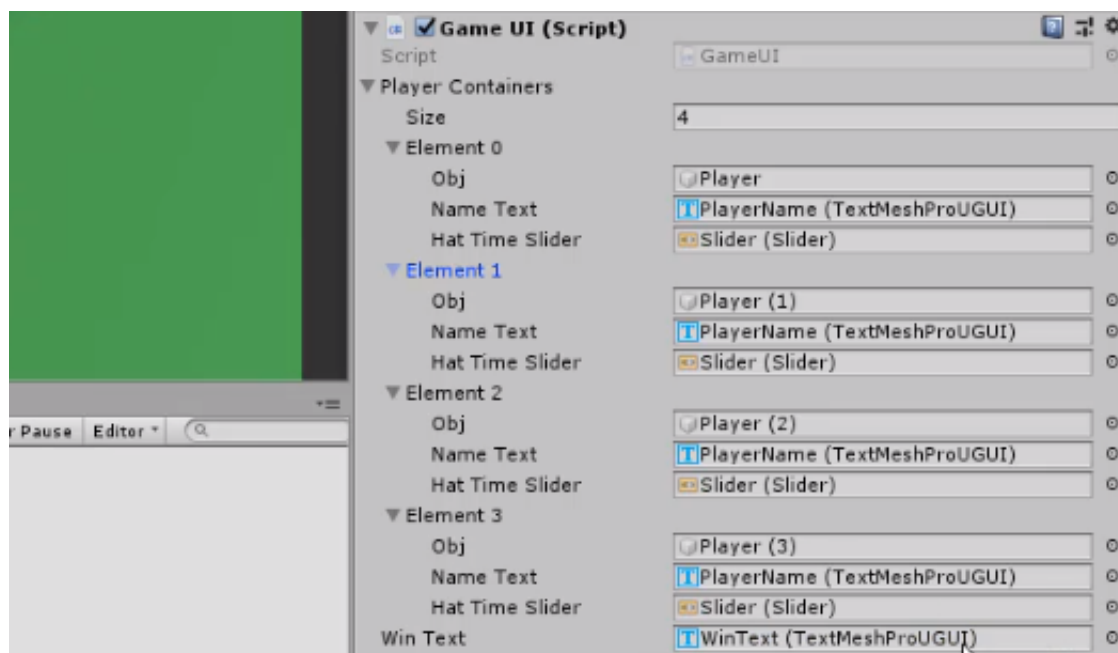
Connecting the UI

In the **GameManager** script let's connect the win text function in the **WinGame** function.

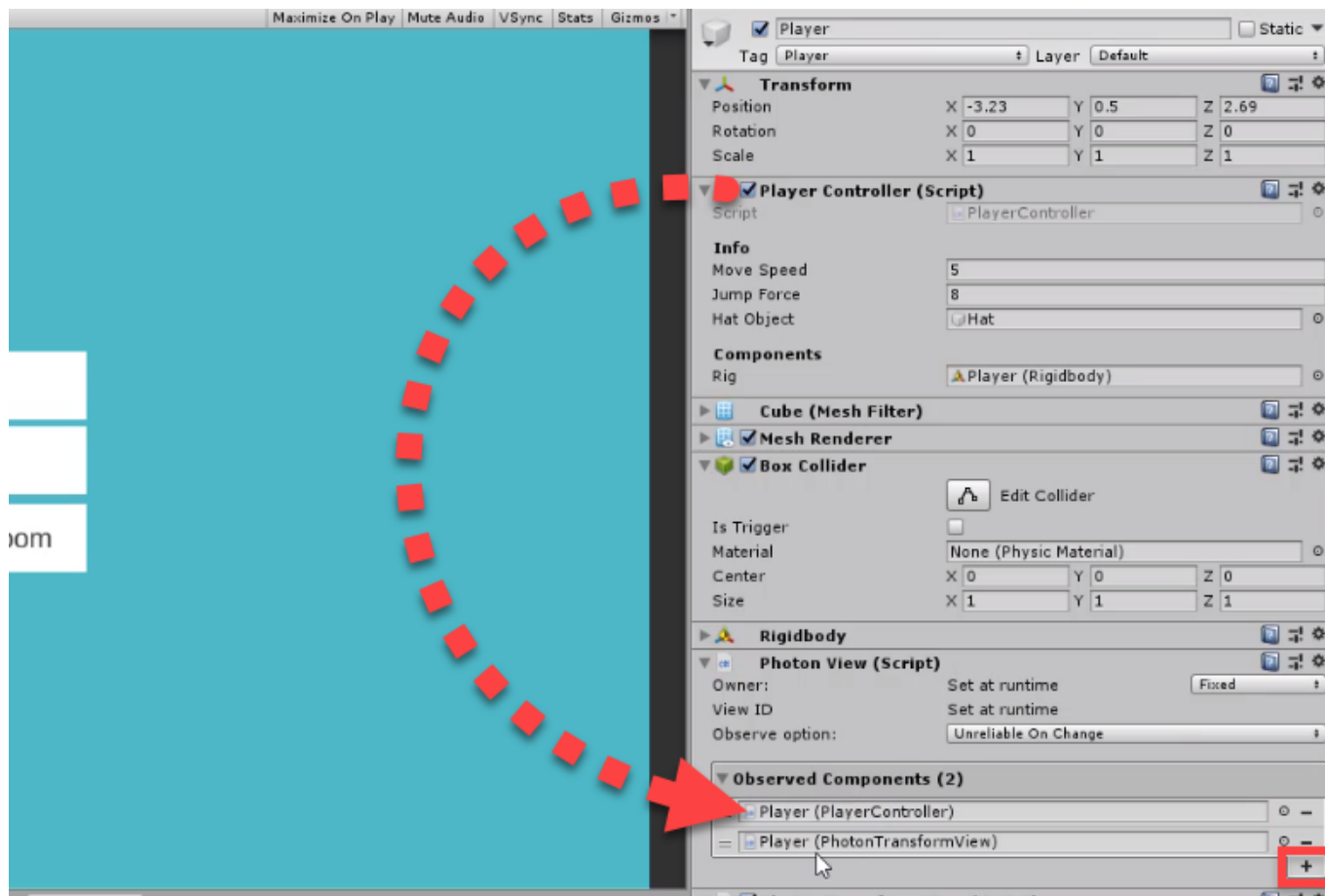
```
GameUI.instance.SetWinText(player.photonPlayer.NickName);
```


Back in the Editor

Back in the editor, let's connect our UI elements to the script.



Let's also select the **Player** prefab and add a new observable component field. Then drag in the **Player Controller** component.





Congratulations on Finishing the Course

We created a fully working multiplayer game in Unity, using Photon. Let's see what we made:

- A lobby system for players to get together before starting the game
- A versatile player controller
 - Movement
 - Jumping
 - Stealing the hat from other players
- We used Unity's UI system